university of groningen

faculty of science and engineering

Unstructured Finite-Volume Discretisation Scheme for Turbulent Navier-Stokes

Master Project Applied Mathematics

24 June 2026

Student: M.A. Mandeville

First Supervisor: prof. dr. ir. R.W.C.P Verstappen

Second Supervisor: dr. ir. R. Luppes

Abstract

In this paper, we present a Finite-Volume discretisation scheme for two- and three-dimensional unstructured grids obtained by a Delaunay triangulation. This triangulation is chosen such the circumcircle of a triangle only contains the vertices of that triangle, and equivalently for tetrahedra. We first go over the basics of turbulence modelling, such as Large Eddy Simulation and filtering. With the aim of the divergence and gradient being adjoints of one another, we present the derivations for the discrete operators, and we give an example for a simple two-dimensional grid. We solve the pressure via a Poisson problem where the intermediate linear systems are solved iteratively. We verify the code using the method of manufactured solutions on a regular mesh, in which we present velocity and pressure solutions as linear combinations of sines and cosines and aim to recover these solutions.

Keywords: Finite-Volume Discretisation, Unstructured Grid, Delaunay Triangulation, Navier-Stokes Turbulence, Large Eddy Simulation, Code verification, Method of Manufactured Solutions.

Contents

1 Introduction **3**
- 1.1 Motivation . . . 3
- 1.2 Mathematical model . . . 3
- 1.3 Outline . . . 4

2 A brief introduction to Large Eddy Simulation **4**
- 2.1 Filtering . . . 5
- 2.2 Filtered Navier-Stokes . . . 6

3 Domain discretisation **6**
- 3.1 Two-dimensional triangulation . . . 7
- 3.2 Three-dimensional tetrahedralisation . . . 7
- 3.3 Relation with rectangular mesh . . . 8
- 3.4 Boundary faces . . . 8
- 3.5 Regular mesh . . . 9
- 3.6 Notation and discrete scalar products . . . 10

4 Deriving the discrete operators **11**
- 4.1 Duality . . . 11
- 4.2 Divergence . . . 11
- 4.3 Gradient . . . 12
- 4.4 Laplacian . . . 12
- 4.5 Convection . . . 13
- 4.6 Two-dimensional example . . . 13
- 4.7 Interpolation . . . 14
- 4.8 Boundary conditions . . . 15

5 Time marching **16**
- 5.1 Pressure Poisson problem . . . 16

6 Numerical implementation **17**
- 6.1 Julia code . . . 17

7 Code verification **19**
- 7.1 Checking the duality condition . . . 19
- 7.2 Method of Manufactured Solutions . . . 20

8 Conclusion and discussion **25**

9 References **27**

A Reynolds equations **28**

B Energy cascade **28**
- B.1 Kolmogorov's first hypothesis . . . 29
- B.2 Kolmogorov's second hypothesis . . . 30
- B.3 Energy spectrum . . . 30

C More details on Large Eddy Simulation **31**
- C.1 Examples of filters . . . 31
- C.2 The Smagorinsky model . . . 32

D Additional plots **35**
- D.1 Method of Manufactured Solutions . . . 35

1 Introduction

1.1 Motivation

There are many examples of turbulent flows around us. Smoke rising from a chimney or a cigarette, water flowing along a river or down a waterfall are some of many examples of turbulent flows that one can observe. If you look at a waterfall in greater detail, and you try to follow the motion of a single droplet of water, you may notice that it seems irregular, random and unpredictable. If you film the motion of smoke coming out of a chimney, you'll notice eddies[1] forming, with a length scale comparable to the width of the chimney. And, as you zoom in, you'll also observe eddies up to the smallest scale that the camera can capture. These observations are common to all turbulent flows. Another important characteristic of these flows is that the change in velocity is significant and irregular both in time and space [12].

We can also describe turbulent flows mathematically. The Reynolds number is a dimensionless number that characterises all flows, defined by

$$\mathrm{Re} = \frac{\mathcal{U}\mathcal{L}}{\nu}, \tag{1}$$

where $\mathcal{U}, \mathcal{L}$ are respectively the characteristic velocity and length of the flow, and ν is the kinematic viscosity. It was first introduced by Stokes in 1851, and later popularised by Reynolds (after whom it was named) in 1883. At low Reynolds number, the flow is typically laminar, meaning that the fluid particles follow smooth paths, and there are no currents perpendicular to the flow and no eddies. For high Reynolds numbers, the flow is turbulent. In between the two, for moderate Reynolds numbers, we are in a transition phase to turbulence. The specific value for which we have turbulence is problem specific. For instance, in the case of a pipe flow, it can be found experimentally that the flow is laminar for $\mathrm{Re} < 2300$ and turbulent for $\mathrm{Re} > 4000$. This range for the Reynolds number is often taken as reference to determine if a flow is laminar or turbulent, but it should not be taken as ground truth. For instance, in a channel flow the critical Reynolds number (i.e. when the flow starts to become turbulent) is about 2000, so similar to that of the pipe flow. However, for e.g. a Couette flow (a flow between two place surfaces, one of which is moving tangentially to the other), the critical Reynolds number is about 1000 [9].

Turbulent flows make up the vast majority of fluid flows, therefore it is generally of interest to model them accurately and to study and understand their properties. Furthermore, turbulent flows transport and mix fluids much more effectively than laminar flows. In many applications, such as the release of pollutants in the atmosphere or water, it is preferred for the mixing to happen as quickly as possible. [9]. Moreover in turbulent flows, drag, heat and mass transfers are increased at the solid-fluid interface, where the interest then lies in determining the material properties of e.g. an aircraft wing.

To this day still, turbulence is considered one of the most challenging problems in fluid dynamics. For instance, we do not know how to predict the Reynolds number of transition in a pipe flow, despite knowing it experimentally, and despite having more powerful computers now than a hundred years ago. We know that we can use numerical schemes to solve the equations of motions, given an initial state and boundary conditions, thanks to the meteorologist Richardson [13, 15]. However, this method of directly solving the equations of motions only really works for laminar flow, and becomes generally far too expensive in practice for high Reynolds numbers. For instance, consider an simulation with $\mathrm{Re} = 1,500$, which is a relatively low value for the Reynolds number. Using the direct method, this simulation can take days to run, and even then only product marginally low quality results [12] Hence, we aim to find a way to obtain sufficiently good resolution, at a comparatively low computational cost.

1.2 Mathematical model

All flows can be modelled by the well-known Navier Stokes equations, written here in their dimensionless form,

$$\frac{\partial \boldsymbol{u}}{\partial t} + \boldsymbol{u} \cdot \nabla \boldsymbol{u} = -\nabla p + \frac{1}{\mathrm{Re}} \nabla^2 \boldsymbol{u}, \tag{2}$$

where $\boldsymbol{u}(\boldsymbol{x}, t)$ is the velocity field and $p(\boldsymbol{x}, t)$ the pressure. However, in the context of turbulence flows, the Navier-Stokes equations describe every detail of the velocity field, at all scales. Since a turbulent velocity field

[1] An eddy is a current of fluid – gas or liquid – that flows against the main current.

contains a large amount of information, namely eddies at all scales, the direct approach of solving the Navier-Stokes is not feasible. This direct approach is called Direct Numerical Simulation (DNS) and is only of interest for moderately low Reynolds numbers. It is conceptually the simplest approach, however its computational cost is extremely high, to the point that for sufficiently large Reynolds number, this becomes impossible [12].

Other models exist for solving the Navier-Stokes equations in a turbulent context, such as the $k-\varepsilon$ model, Reynolds-stress models and probability based models. In the following, we focus on one such model, Large Eddy Simulation. This model is presented in Section 2, and some further details are given in Appendix C. We give a brief introduction to Reynolds stress models in Appendix A, in which we present the Reynolds equations and the main idea behind the model. A further analysis and comparison of the different models can be found in [12].

Before looking into the specifics of turbulence modelling, we first recall the equations for incompressible fluids, given by the continuity equation

$$\nabla \cdot \boldsymbol{u} = 0 \tag{3}$$

for the conservation of mass, and by the Navier-Stokes equations (2) for the conservation of momentum. Equivalently, these equations can be written out component-wise,

$$\frac{\partial u_j}{\partial x_j} = 0, \tag{4a}$$

$$\frac{\partial u_i}{\partial t} + \frac{\partial}{\partial x_j} u_i u_j = -\frac{\partial p}{\partial x_i} + \frac{1}{\mathrm{Re}} \frac{\partial}{\partial x_j} \left(\frac{\partial u_i}{\partial x_j} + \frac{\partial u_j}{\partial x_i} \right), \tag{4b}$$

where we make use of the shorthand sum notation, in which the sum is to be taken over repeating indices, where here the sum is over j.

1.3 Outline

This paper is structured as follows. Section 2 gives a brief introduction to turbulence modelling and Large Eddy Simulation (LES). Section 3 introduces Delaunay triangulation to discretise the spatial domain. Notation is introduced and examples in two- and three-dimensions are given. The discrete operators such as divergence, gradient, Laplacian and convection are derived in Section 4, in which a simple two-dimensional example is used for illustrative purposes. A time marching procedure, which uses an explicit Euler discretisation, is given in Section 5, in which the pressure and velocity are updated by use of an intermediate velocity and a Poisson problem. Section 6 goes over the numerical implementation of both spatial and temporal discretisations. The method of manufactured solutions is also used as a mean to verify the code in Section 7. Finally, Section 8 presents a conclusion and a discussion as to some of the further work than can be investigated. Further details on turbulence modelling are introduced in Appendices A, B, C, and additional plots are given in Appendix D.

2 A brief introduction to Large Eddy Simulation

As mentioned in the introduction directly solving the Navier-Stokes equations for high Reynolds number is unfeasible. The computational cost is exceedingly high, for a marginally good result. For instance, Pope [12] gives a few estimates of runtime required for different Reynolds numbers and different grid resolutions for the Direct Numerical Method – which is the method consisting of directly solving the equations of motion. We have highlighted a few of the values in Table 1. Note that the time estimates are already for a computer more powerful than a standard laptop from today.

In the last hundred years, there have been computationally cheaper ways to solve the equations of motions for turbulent flows. One such way is via Reynolds averaging and Reynolds stress models [12], where Reynolds averaging is discussed further in Appendix A. In this paper, we focus on another method, namely Large Eddy Simulation (LES). In terms of computational cost, LES is between DNS and Reynolds stress models, but it is expected to be more accurate and reliable than the latter. This is due to the fact that, as the name implies, the large scale motions are represented explicitly. Furthermore, compared to DNS, we avoid the high cost associated with the smaller scale motions.

Re	N	Time
94	104	minutes
375	214	hours
1500	498	days
6000	1260	months
24000	3360	years

Table 1. Time estimates for Direct Numerical Simulation (DNS) at various Reynolds numbers with N nodes needed in each spatial direction [12, Chapter 9].

A greater detail on the different scales of motions and the transfer of energy between them is given in Appendix B. The main idea that should be taken from it is that the large scale unsteady turbulent motions (eddies) break up into smaller motions. However, these smaller motions are still unsteady, so they break up into even smaller motions, and so on. As the motions break up, energy is transferred from the larger scales to the smaller. The process ends at the smallest scale of motion, called the Kolmogorov scale, where energy is dissipated into heat.

Large Eddy Simulation aims to resolve the larger scales, and to this end, we consider the following four conceptual steps [12].

1. Filtering, in which we decompose the velocity
$$\boldsymbol{u}(\boldsymbol{x},t) = \overline{\boldsymbol{u}}(\boldsymbol{x},t) + \boldsymbol{u}'(\boldsymbol{x},t), \tag{5}$$
where $\overline{\boldsymbol{u}}$ is the filtered or resolved velocity, and $\boldsymbol{u}'$ is the residual or subgrid scale velocity. The filtered velocity field represents the motion of the large eddies.
2. The equations for the evolution of $\overline{\boldsymbol{u}}$ are derived from applying the filtering operation to the Navier-Stokes equations. A residual stress tensor arises from the residual velocity $\boldsymbol{u}'$.
3. This stress tensor produces a closure problem, in which there are more unknowns than equations. This is solved by modelling the stress tensor.
4. The filtered equations are solved numerically for $\overline{\boldsymbol{u}}$, giving an approximation for the large scale motions in a turbulent flow.

Then, from applying a filtering operation, we can see that in LES, the larger unsteady motions are directly represented. On the other hand, the effects of the smaller scales of motions are modelled through the residual stress tensor. Each of these steps are briefly detailed below, and in more detail in Appendix C.

2.1 Filtering

We apply a spatial filtering operation to the velocity $\boldsymbol{u}$, so that the resulting filtered velocity $\overline{\boldsymbol{u}}$ can be resolved on a relatively coarse grid. The filter is characterised by a specified filter length Δ, and in an ideal case this length is to be taken less than ℓ_{EI} [12], which is the size of the smallest energy containing motions, as defined in Appendix B.1. The general filtering operation is defined by the convolution
$$\overline{\boldsymbol{u}}(\boldsymbol{x},t) = \int G(\boldsymbol{r},\boldsymbol{x})\boldsymbol{u}(\boldsymbol{x}-\boldsymbol{r},t)\,\mathrm{d}\boldsymbol{r}, \tag{6}$$
where we integrate over the whole flow domain. Furthermore, the specified filter function G is normalised,
$$\int G(\boldsymbol{r},\boldsymbol{x})\,\mathrm{d}\boldsymbol{r} = 1. \tag{7}$$
In the simplest case, the filter G is independent of $\boldsymbol{x}$. In the remaining of this paper, we assume that the filter is given implicitly by the grid discretisation. The grid discretisation is further analysed in Section 3. Other examples of filters – namely the box, sharp cutoff and Gaussian filters – are given in Appendix C.1. All filters considerer here are spatially uniform, which means that they commute with differentiation [12].

2.2 Filtered Navier-Stokes

After having defined the filtering operation and the filtered and residual velocity fields, we can apply the filter to the Navier-Stokes equation, to obtain the equations of motion of the filtered velocity. We first have for the continuity equation (4a)

$$\overline{\frac{\partial u_j}{\partial x_j}} = \frac{\partial \overline{u}_j}{\partial x_j} = 0, \tag{8}$$

where we make use of the assumption that filtering commutes with differentiation, for the choice of filter made here. This then yields the filtered continuity equation. For the residual continuity equation we have

$$\frac{\partial u'_j}{\partial x_j} = \frac{\partial}{\partial x_j}(u_j - \overline{u}_j) = 0. \tag{9}$$

So, both the filtered and residual velocity fields are divergence free. Applying the filtering operation to the momentum equation (4b) yields

$$\frac{\partial \overline{u}_i}{\partial t} + \frac{\partial \overline{u_i u_j}}{\partial x_j} = -\frac{\partial \overline{p}}{\partial x_i} + \frac{1}{\mathrm{Re}} \frac{\partial^2 \overline{u}_i}{\partial x_j^2}, \tag{10}$$

where $\overline{p}(\boldsymbol{x}, t)$ is the filtered pressure field. In general, the filtered product of velocities is not the product of the filtered velocities, that is in general $\overline{u_i u_j} \neq \overline{u}_i \overline{u}_j$. We denote their difference by the residual stress tensor

$$\tau_{ij}^R := \overline{u_i u_j} - \overline{u}_i \overline{u}_j. \tag{11}$$

From this, we define the anisotropic residual stress tensor [12]

$$\tau_{ij}^r := \tau_{ij}^R - \frac{2}{3} k_r \delta_{ij}, \tag{12}$$

where $k_r := \frac{1}{2}\tau_{ii}^R$ is the residual kinetic energy. The kinetic energy and the filtered pressure can be combined into the modified pressure – still denoted here by $\overline{p}$ as done in [12] – such that the filtered momentum equations can be written as

$$\frac{\partial \overline{u}_i}{\partial t} + \frac{\partial \overline{u}_i \overline{u}_j}{\partial x_j} = -\frac{\partial \overline{p}}{\partial x_i} + \frac{1}{\mathrm{Re}} \frac{\partial^2 \overline{u}_i}{\partial x_j} - \frac{\partial \tau_{ij}^r}{\partial x_j}. \tag{13}$$

Then, this gives the equations of motion for the filtered velocity field as

$$\nabla \cdot \overline{\boldsymbol{u}} = 0, \tag{14a}$$

$$\frac{\partial \overline{\boldsymbol{u}}}{\partial t} + \overline{\boldsymbol{u}} \cdot \nabla \overline{\boldsymbol{u}} = -\nabla \overline{p} + \frac{1}{\mathrm{Re}} \nabla^2 \overline{\boldsymbol{u}} - \nabla \cdot \boldsymbol{\tau}. \tag{14b}$$

As mentioned at the start of this section, closure for the filtered equations is achieved by modelling the stress tensor τ_{ij}^r. One such way is the Smagorinsky model, further detailed in Appendix C.2. This model is the simplest model for the residual stress tensor.

However, in the remaining of the paper, for simplicity we disregard this term. In other words, together with the fact that we consider an implicit filter from the mesh, we consider the following governing equations

$$\nabla \cdot \boldsymbol{u} = 0, \tag{15a}$$

$$\frac{\partial \boldsymbol{u}}{\partial t} + \boldsymbol{u} \cdot \nabla \boldsymbol{u} = -\nabla p + \frac{1}{\mathrm{Re}} \nabla^2 \boldsymbol{u}, \tag{15b}$$

where we also dropped the overline notation, since no explicit filter as being used. This also makes it clearer in the event that an explicit filter is used in addition to the mesh filtering. For instance, when considering a regularisation of the convective operator in Section 4.5.

3 Domain discretisation

As mentioned in the previous section, we here use an implicit filter determined by the grid discretisation. In this section, we consider a specific spatial discretisation, namely the Delaunay triangulation. This triangulation is chosen for its relatively easy and well-known implementation, and for its interesting properties. Namely, it offers straightforward primal and dual meshes. We illustrate the triangulation both in two- and three-dimensions, and we also relate it to the more commonly used rectangular grid. We also set up the notation and discrete scalar products, which is then used in Section 4 when deriving the discrete divergence and gradient operators.

3.1 Two-dimensional triangulation

We apply a Delaunay triangulation to a two-dimensional flow domain Ω, to obtain a primal grid. This triangulation was first introduced by Dirichlet as a way to connect an arbitrary set of points together to produce a unique set of triangles, in such a way that the vertices of a triangle do not lie on or inside the circumcircles of other triangles [4]. This allows to avoid triangles with two very acute angles, as well as it allows us to define a natural dual grid. This dual grid is natural in the sense that it follows rather straightforwardly from the primal grid, by connecting the circumcentres. In fact, this dual grid is called a Voronoi diagram, and is sometimes used to derive a Delaunay triangulation. Such primal/dual grids and the primal circumcircles are illustrated in Figure 1. Since the dual grid is obtained by connecting the circumcentres, it follows that each dual edge crosses a primal edge perpendicularly and in the middle of the primal edge.

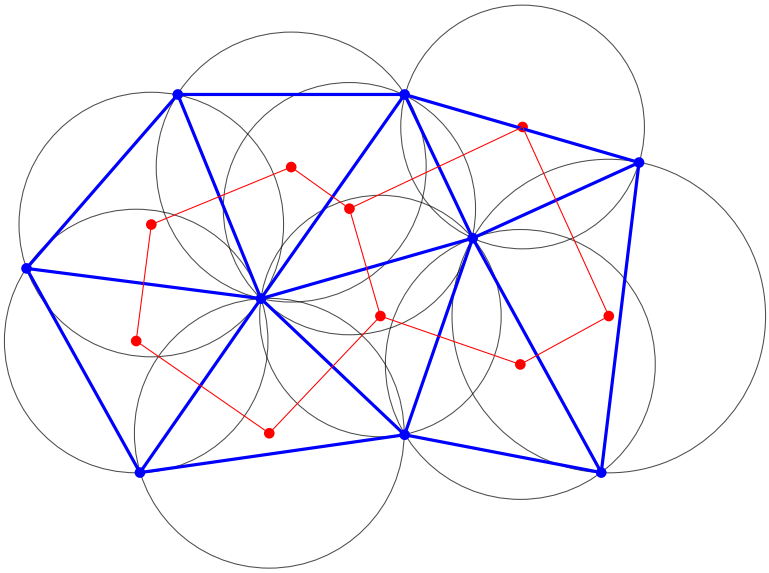


Figure 1. An example of a primal Delaunay triangulation (thick blue) and the corresponding dual Voronoi diagram (fine red). The circumcircles to the primal cells are also drawn (fine grey) where it can be thus seen that each circle only goes through three vertices, and no vertices lie inside of it.

3.2 Three-dimensional tetrahedralisation

The previously seen triangulation has the property that it can extend quite straightforwardly to three-dimensions. Instead of having triangles as cells and lines as faces, we now have tetrahedra and triangles, as illustrated in Figure 2, where for the sake of clarity only two cells with a common face are drawn. The dual edges still consist of lines, and by virtue of once again connecting the circumcentres, we have that they intersect the primal faces perpendicularly and in their centre. Centre here refers to the circumcentre of the triangular face.

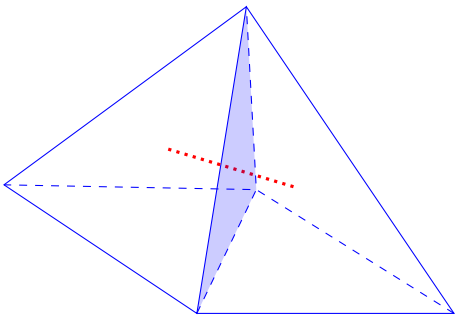


Figure 2. Two tetrahedra with a common face (solid and dashed blue) and the corresponding dual edge (dotted red) which lies fully inside the two cells.

3.3 Relation with rectangular mesh

By considering rectangles instead instead of triangles, we can also define a dual grid by connecting the corresponding circumcentres. This is illustrated in Figure 3. Here too the circumcircles only contain the vertices of the respective primal cell.

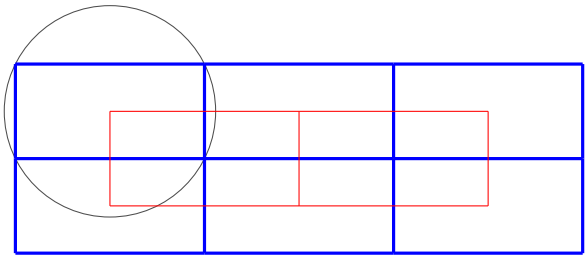


Figure 3. The rectangular primal and dual grids, respectively thick blue and fine red. The dual grid is obtained by connecting the centres of the circumcircles. One such circumcircle is drawn in grey, where it can be seen that the circle only goes through the four vertices of that cell, and no other vertex of the primal grid lies inside of it.

3.4 Boundary faces

From the above examples, one may question whether a dual edge $\hat{e}$ is defined when considering a primal face e which lies on the boundary of the domain. The answer is yes; a dual edge can be obtained by considering the symmetry of the circumcenter with regards to face e. This can be seen as having a mirror of the primal cell, or in other words a ghost cell, for which we take the circumcentre. An illustration is provided in Figure 4

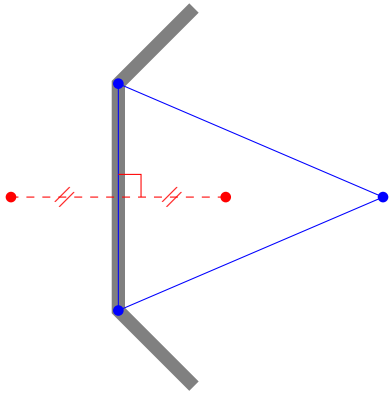
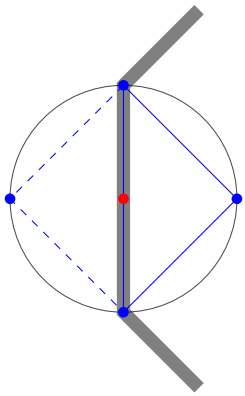


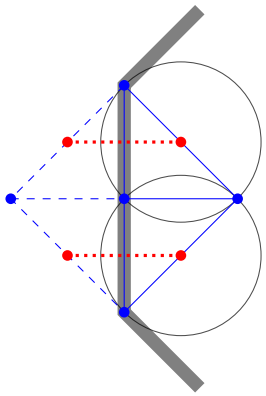
Figure 4. A primal grid cell (solid blue) sharing a face with the domain boundary (thick grey). The corresponding dual edge (dashed red) is obtained by axial symmetry of the primal circumcentre with regards to the boundary face.

In the case that the circumcentre lies exactly on the boundary face, then this is not quite a valid Delaunay triangulation. Indeed, in that case, the ghost mirror cell shares the same circumcircle, and thus there are four vertices which lie on the circumcircle. The way to proceed is then to add a vertex and a face, to split the primal cell into two. Then, the two boundary faces are dealt in the same way as described above and in Figure 4. An illustration to this scenario is also given in Figure 5. Note that in this figure, after having cut the cell into two, the circumcentres of the new cells are also on the faces. If the circumcircles also go through or contain other vertices, then this is not a valid Delaunay triangulation, and more vertices need to be added until a valid triangulation is obtained. Further detail on algorithms for Delaunay triangulations can be found in [3, 4].

Note that the newly placed vertex does not need to be where the circumcentre was, it only needs to be on the boundary. The easiest choice nonetheless is to place it there.



(a) A primal grid cell (solid blue) sharing a face with the domain boundary (thick grey), for which the circumcentre (red) is located on the boundary face. The mirror ghost cell is also drawn (dashed blue), showcasing that there are four vertices lying on the same circumcircle (grey).



(b) The primal cell from **(a)** divided into two by adding a vertex and a face (solid blue) The respective circumcircles (grey) and circumcentres (red dot) are also draw). The two ghost cells (dashed blue) and circumcircles (red dot) are given, highlighting the fact that the dual edges (dotted red) are nonzero.

Figure 5. Illustration of the treatment of the dual edge for boundary faces in the case where the circumcentre lies exactly on the boundary face.

3.5 Regular mesh

In order to be able to implement mesh refinement in a consistent manner, we implement a regular triangular mesh. This mesh still follows the principles of the Delaunay triangulation aforementioned. The regularity here comes from having identical equilateral triangles for the primal grid. This is illustrated in Figure 6 in two dimensions. For the three-dimensional regular tetrahedral mesh, the same reasoning applies, where we instead have regular tetrahedra for the primal cells.

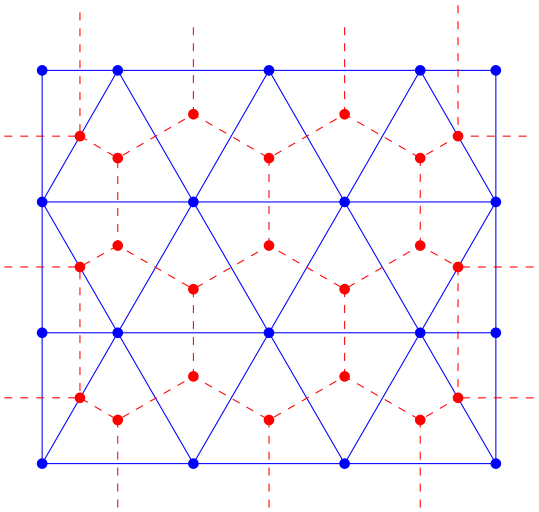


Figure 6. A regular triangulation of $\Omega = [0,1] \times [0, \frac{\sqrt{3}}{2}]$, with $N = 4$ nodes on the x-axis. Both the primal (solid blue) and dual (dashed red) are drawn, where for the dual grid we have applied the extension for the boundary faces, as explained in Section 3.4.

The regular mesh is defined in terms of the unique parameter N, which determines how many nodes are one the sides of the rectangular domain. For simplicity, when considering a numerical implementation, we use the domain $\Omega = [0,1] \times [0, \frac{\sqrt{3}}{2}]$, where $\frac{\sqrt{3}}{2}$ is the height of an equilateral triangle with side length 1. This allows us to have the same number of nodes on all sides. Then, increasing N leads to a grid refinement procedure.

3.6 Notation and discrete scalar products

The notation used for dual and primal cells and faces is given in Table 2, which is adapted from [8]. The notation is consistent with all aforementioned examples and illustrations, were the colouring for primal and dual cells, respectively blue and red was kept for clarity. In the remainder of this paper, the dual cells are not considered, but their notation is added for the sake of completeness. Furthermore, when used, $\hat{e}$ refers to the dual edge that crosses the primal edge e perpendicularly.

Symbol	Meaning	Symbol	Meaning
K, L	cells of primal grid	$\hat{K}, \hat{L}$	cells of dual grid
$e := K \mid L$	face between primal cells K and L	$\hat{e} := \hat{K} \mid \hat{L}$	edge between dual cells $\hat{K}$ and $\hat{L}$
$\lvert e\rvert, \lvert\hat{e}\rvert$	length/area of face	$\lvert K\rvert, \lvert\hat{K}\rvert$	area/volume of cell
$\partial K, \partial\hat{K}$	set of cell edges	d	dimension of flow domain Ω
$\mathcal{C}$	set of primal cells	n_c	number of cells
$\mathcal{F}$	set of primal faces	n_f	number of faces

Table 2. Notation used for the primal and dual grids.

We denote $\mathcal{C}$ the set of primal cells and $\mathcal{F}$ the set of primal faces, n_c, n_f to be the number of primal cells and faces respectively, and d the dimension of the domain considered (either 2 or 3). We denote $\boldsymbol{p}_c \in \mathbb{R}^{n_c}$ the pressure evaluated at the cell-centres and $\boldsymbol{u}_c \in \mathbb{R}^{n_c \times d}$ the cell-centred velocity in each of the d components, and $\boldsymbol{u}_f \in \mathbb{R}^{n_f}$ the face-centred velocity. Then, for instance, the velocity and pressure at the K-th cell are respectively $p_K \in \mathbb{R}$ and $\boldsymbol{u}_K \in \mathbb{R}^d$, where the components of the velocity vectors correspond to each spatial direction. The aforementioned cell-centred notation is illustrated in Figure 7.

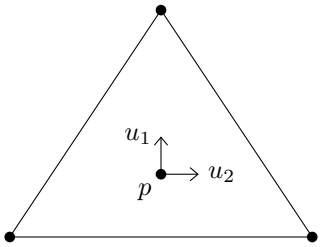


Figure 7. The cell-centred velocity and pressure on one cell, where the centre corresponds to the cell's circumcentre and the velocity is given in each direction.

After having introduced the notation for cells and faces, we can define discrete inner products on them. These scalar products were introduced by Korn and Linardakis [8], where more information on them as well as on the choice of notation can be found in their paper. Here, we simply state them, and make use of them when deriving the discrete gradient and convection operators in Section 4. The inner products are defined as sums over the cells/faces, weighted by the respective volumes, that is

$$\langle \boldsymbol{u}_f, \boldsymbol{v}_f \rangle_{\mathcal{F}} := \sum_{e \in \mathcal{F}} |e|\,|\hat{e}|\,(\boldsymbol{u}_f)_e (\boldsymbol{v}_f)_e, \tag{16a}$$

$$\langle \boldsymbol{p}_c, \boldsymbol{q}_c \rangle_{\mathcal{C}} := \sum_{K \in \mathcal{C}} |K|\,(\boldsymbol{p}_c)_K (\boldsymbol{q}_c)_K, \tag{16b}$$

$$\langle \boldsymbol{u}_c, \boldsymbol{v}_c \rangle_{\mathcal{C}} := \sum_{K \in \mathcal{C}} |K|\,(\boldsymbol{u}_c)_K \cdot (\boldsymbol{v}_c)_K, \tag{16c}$$

where the last two scalar products on the cells (16b) and (16c) only differ in the fact that the pressure and the velocity on a cell are respectively a scalar and a d-dimensional vector, and hence the Euclidean scalar product needs to be taken in the latter case. In this paper, when it is clear from context whether we mean cell-centred or face-centred velocity, we omit the c and f subscripts, i.e. we write $\boldsymbol{u}_K := (\boldsymbol{u}_c)_K$ and $u_e = (\boldsymbol{u}_f)_e$.

Furthermore, each face e has a natural outward normal vector $\boldsymbol{n}_e$, which is independent of the neighbouring cells. Since when deriving the divergence on a cell K, we aim to have the outward normals to this cell, we define

an indicator function

$$n_{e,K} = \begin{cases} +1 & \text{if } \boldsymbol{n}_e \text{ is outward to } K, \\ -1 & \text{if } \boldsymbol{n}_e \text{ is inward to } K. \end{cases} \tag{17}$$

This normal indicator is illustrated in Section 4.6, when considering a simple two-dimensional example for the discrete operators defined in the next section.

4 Deriving the discrete operators

In this section we aim to derive the discrete divergence and gradient operators, making use of their duality property, which is also derived. From these operators, we obtain the Laplacian and the convection. We then illustrate the gradient and divergence with a simple two-dimensional example. We also define the face-to-cell and cell-to-face interpolating functions, which are needed when implementing the time marching (Section 5) as well as for the boundary conditions.

4.1 Duality

We first briefly stick to the continuous case. Consider a d-dimensional flow domain Ω with boundary $\partial\Omega$ and suppose that either the pressure $p \in \mathbb{R}$ or the velocity $\boldsymbol{u} \in \mathbb{R}^d$ vanish at the boundary. Then, integrating $\nabla \cdot (p\boldsymbol{u})$ over the domain yields

$$\int_\Omega \nabla \cdot (p\boldsymbol{u}) \,\mathrm{d}\boldsymbol{x} = \int_\Omega \nabla p \cdot \boldsymbol{u} \,\mathrm{d}\boldsymbol{x} + \int_\Omega p \nabla \cdot \boldsymbol{u} \,\mathrm{d}\boldsymbol{x} \tag{18}$$

Applying the divergence theorem to the left hand side term gives

$$\implies \int_{\partial\Omega} p\boldsymbol{u} \cdot \boldsymbol{n} \,\mathrm{d}\boldsymbol{s} = \int_\Omega \nabla p \cdot \boldsymbol{u} \,\mathrm{d}\boldsymbol{x} + \int_\Omega p \nabla \cdot \boldsymbol{u} \,\mathrm{d}\boldsymbol{x} \tag{19}$$

And since we assume that either p or $\boldsymbol{u}$ vanish at the boundary, the left hand side is zero, hence

$$\implies \int_\Omega \nabla p \cdot \boldsymbol{u} \,\mathrm{d}\boldsymbol{x} = -\int_\Omega p \nabla \cdot \boldsymbol{u} \,\mathrm{d}\boldsymbol{x}, \tag{20}$$

Which thus shows that $\langle \nabla p, \boldsymbol{u} \rangle = \nabla p, -\nabla \cdot \boldsymbol{u}$, or in other words,

$$\mathrm{grad}^* = -\,\mathrm{div}\,. \tag{21}$$

That is, the adjoint of the gradient is minus the divergence. This is used when deriving the discrete gradient in Section 4.3.

4.2 Divergence

To define the discrete divergence, we first define the divergence of $\boldsymbol{u}$ on cell K as a normalised integral, then we use the divergence theorem and discretise the integral using a Finite-Volume approach,

$$\begin{aligned} \mathrm{div}\, \boldsymbol{u}_K &:= \frac{1}{|K|} \int_K \nabla \cdot \boldsymbol{u} \,\mathrm{d}\boldsymbol{x} \\ &= \frac{1}{|K|} \int_{\partial K} \boldsymbol{u} \cdot \boldsymbol{n} \,\mathrm{d}\boldsymbol{s} \\ &= \frac{1}{|K|} \sum_{e \in \partial K} u_e n_{e,K} \,|e|\,, \end{aligned} \tag{22}$$

where in the second line the normal vectors $\boldsymbol{n}$ are taken to be *outward* to cell K, and $u_e = \boldsymbol{u}_e \cdot \boldsymbol{n}_e$ is defined to be the face-centred velocity in the normal direction $\boldsymbol{n}_e$ (which could very well be inward to K). This is hence where the normal indicator $n_{e,K}$ as defined in equation (17) is required; to guarantee that all face-centred velocities are taken in the outward direction with regards to cell K. We have then derived the face-to-cell discrete divergence operator (22).

4.3 Gradient

Using the duality relation (21), the discrete inner products on cells (16b) and faces (16a) and the discrete divergence (22), we obtain:

$$\begin{aligned} \langle \boldsymbol{u}, \operatorname{grad} p \rangle_{\mathcal{F}} &= -\langle \operatorname{div} \boldsymbol{u}, p \rangle_{\mathcal{C}} \\ \implies \sum_e |e|\,|\hat{e}|\,u_e \operatorname{grad} p_e &= -\sum_K |K|\,(\operatorname{div} \boldsymbol{u}_K) p_k \\ &= -\sum_K p_K \sum_{e \in \partial K} u_e n_{e,K}\,|e| \end{aligned} \tag{23}$$

The left hand side consists in a sum over all faces, and the right hand side in a double sum of all cells and all faces per cell. Then, we can rewrite the right hand side as a sum over each face, where all faces are counted twice for both neighbouring cells (and once for the boundary faces). Looking at one face in particular, between cells K and L, $e = K|L$ we have

$$\implies |e|\,|\hat{e}|\,u_e \operatorname{grad} p_e = -u_e\,|e|\,(p_K n_{e,K} + p_L n_{e,L}). \tag{24}$$

For boundary faces e, we proceed with the same treatment as in Section 3.4, where we then need to implement some boundary condition (which is detailed further in Section 4.8). We can simplify equation (24) by noting that if $\boldsymbol{n}_e$ is outward to K, then it must be inward to L, and vice versa, hence $n_{e,K} = -n_{e,L}$. This then yields the discrete cell-to-face gradient operator

$$\operatorname{grad} p_e = \frac{p_L - p_K}{|\hat{e}|} n_{e,K}. \tag{25}$$

4.4 Laplacian

In a continuous setting, the Laplacian is defined as the divergence of the gradient, $\nabla^2 := \nabla \cdot \nabla$. This also holds in the discrete sense. If we denote the discrete divergence and gradient operators as $D \in \mathbb{R}^{n_c \times n_f}$ and $G \in \mathbb{R}^{n_f \times n_c}$, where n_c and n_f are the numbers of cells and faces respectively, then we obtain the Laplacian as:

$$L := DG \in \mathbb{R}^{n_c \times n_c}. \tag{26}$$

More specifically, we can write down an explicit form for D and G. Since the divergence is cell-to-face, meaning that its input is a face-centred value and its output is cell-centred, we have that the entry D_{ij} must correspond to cell K_i and face e_j. Then, looking at equation (22), we obtain the following:

$$D_{ij} = \begin{cases} \dfrac{|e_j|}{|K_i|} n_{e_j,K_i}, & \text{if } e_j \in \partial K_i, \\ 0, & \text{else.} \end{cases} \tag{27}$$

Similarly, the gradient is cell-to-face and therefore G_{ij} corresponds to cell K_j and face e_i, so

$$G_{ij} = \begin{cases} -\dfrac{1}{|\hat{e}_i|} n_{e_i,K_j}, & \text{if } e_i \in \partial K_j \\ 0, & \text{else.} \end{cases} \tag{28}$$

It can be seen that due to every face only being between two cells, that both D and G have full column and row rank respectively. This only breaks for invalid Delaunay triangulations, in which there might be some ambiguity surrounding the dual edge. Hence, since $n_c \leq n_f$, it here holds that

$$\operatorname{rank} D = \operatorname{rank} G = n_c \implies \operatorname{rank} L = n_c. \tag{29}$$

In other words, the Laplacian has full rank and therefore it is nonsingular.

Both operators (27) and (28) are illustrated in Section 4.6, when considering a two-dimensional example.

4.5 Convection

The convection of an arbitrary velocity field $\boldsymbol{v}$ in a continuous setting is defined as $\boldsymbol{u}\cdot\nabla\boldsymbol{v}$. Applying the discrete gradient (25) and the discrete face-valued scalar product (16a), we obtain the convection on the cell-centred velocity $\boldsymbol{v}_c$ evaluated at cell K as:

$$C(\boldsymbol{u}_f,\boldsymbol{v}_c)_K = \sum_{e\in\partial K} |\hat{e}|\,|e|\,(\boldsymbol{u}_f)_e(G\boldsymbol{v}_c)_e, \tag{30}$$

where $(\boldsymbol{u}_f)_e$ denotes the face-centred velocity on face e and $(G\boldsymbol{v}_c)_e$ the cell-to-face gradient of $\boldsymbol{v}$ on face e.

4.5.1 Regularisation

The nonlinear convective term relates unphysical and physical cell-centred velocity components to one another, which then leads to some spurious or unphysical results [19]. Namely, it produces some additional small scales motions. One way to counter this is to introduce some regularisation. We here present the regularised convective term from [21], which can be seen as a higher-order approximation of the convection,

$$C_2(\boldsymbol{u}_f,\boldsymbol{v}_c) = \overline{C(\overline{\boldsymbol{u}}_f,\overline{\boldsymbol{v}}_c)}, \tag{31a}$$

$$C_4(\boldsymbol{u}_f,\boldsymbol{v}_c) = C(\overline{\boldsymbol{u}}_f,\overline{\boldsymbol{v}}_c) + \overline{C(\overline{\boldsymbol{u}}_f,\boldsymbol{v}'_c)} + \overline{C(\boldsymbol{u}'_f,\overline{\boldsymbol{v}}_c)}, \tag{31b}$$

$$C_6(\boldsymbol{u}_f,\boldsymbol{v}_c) = C(\overline{\boldsymbol{u}}_f,\overline{\boldsymbol{v}}_c) + C(\overline{\boldsymbol{u}}_f,\boldsymbol{v}'_c) + C(\boldsymbol{u}'_f,\overline{\boldsymbol{v}}_c) + \overline{C(\boldsymbol{u}'_f,\boldsymbol{v}'_c)}. \tag{31c}$$

The notation here is the same as introduced in Section 2.1, such that $\boldsymbol{u}'_f + \overline{\boldsymbol{u}}_f = \boldsymbol{u}_f$ is the decomposition of the velocity into the residual and filtered velocities respectively. A filter is also applied to the convection (30) to obtain $\overline{C}$. Here, a specific filter may be used for the convection, different that the implicit filter from the mesh used previously, as given in e.g. [19, 21]. Then, the above regularised convections restrain the aforementioned production of small scale motions.

In the remainder of this paper, we consider the unregularised convection (30). This regularisation may be used in further work for more accurate results, by reducing the complexity of the model and stopping the production of small scale motions.

4.6 Two-dimensional example

In this section we illustrate the operators defined above with a simple example. Consider the rectangular two-dimensional domain $\Omega \subset \mathbb{R}^2$, triangulated with 5 vertices, where the labelling of vertices, faces and cells is as given in Figure 8.

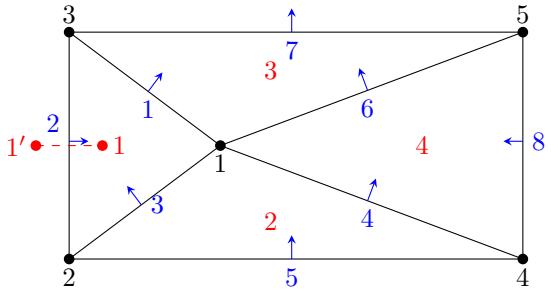


Figure 8. Example of a Delaunay triangulation of a two-dimensional rectangular domain with a labelling of vertices (black), cells (red), and faces with corresponding normal vectors (blue). For the boundary face e_2, the ghost cell was added, as explained Section 3.4.

To illustrate the divergence and gradient operators, we first turn to cell K_1. On this cell, the divergence of the velocity is given by a sum of the face velocities, in the outward normal direction. Face e_1 has normal outward to K_1, however both faces e_2 and e_3 have normal inward to K_1. This is then where the normal indicator function $n_{e,K}$ comes into play, as it allows us to take the face velocities in the outward direction with respect to cell K_1. In other words,

$$\operatorname{div}\boldsymbol{u}_{K_1} = \frac{u_{e_1}|e_1| - u_{e_2}|e_2| - u_{e_3}|e_3|}{|K_1|}, \tag{32}$$

where the minus sign in front of the last two terms comes from $n_{e_i,K_1} = -1$ for $i = 2, 3$.

Similarly, the pressure gradient on face e_1 is given by the difference in pressures between cells K_3 and K_1, since the normal to face e_1 is pointing in the direction of K_3. In other words,

$$\operatorname{grad} p_{e_1} = \frac{p_{K_3} - p_{K_1}}{|\hat{e}_1|}. \tag{33}$$

If we then interest ourselves to the gradient on a boundary face, say e.g. on face e_2, we first need to introduce the mirror ghost cell centre, which is illustrated in Figure 8. From this, it becomes clear that the pressure gradient on this face is given by:

$$\operatorname{grad} p_{e_2} = \frac{p_{K_{1'}} - p_{K_1}}{|\hat{e}_2|}. \tag{34}$$

If, for instance, we wish to impose zero Neumann boundary conditions on this boundary, this would give:

$$p_{K_{1'}} = p_{K_1}, \tag{35}$$

which would be one way to impose boundary conditions in this case. A more in-depth explanation of the boundary conditions is given in Section 4.8.

We can also illustrate the operators for the divergence and the gradient as defined in (27) and (28) respectively. For this example, we have

$$D = \begin{bmatrix} \frac{|e_1|}{|K_1|} & -\frac{|e_2|}{|K_1|} & -\frac{|e_3|}{|K_1|} & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & \frac{|e_3|}{|K_2|} & \frac{|e_4|}{|K_2|} & -\frac{|e_5|}{|K_2|} & 0 & 0 & 0 \\ -\frac{|e_1|}{|K_3|} & 0 & 0 & 0 & 0 & -\frac{|e_6|}{|K_3|} & \frac{|e_7|}{|K_3|} & 0 \\ 0 & 0 & 0 & -\frac{|e_4|}{|K_4|} & 0 & \frac{|e_6|}{|K_4|} & 0 & -\frac{|e_8|}{|K_4|} \end{bmatrix}, \tag{36a}$$

$$G = \begin{bmatrix} -1/|\hat{e}_1| & 0 & 1/|\hat{e}_1| & 0 \\ -1/|\hat{e}_2| & 0 & 0 & 0 \\ 1/|\hat{e}_3| & -1/|\hat{e}_3| & 0 & 0 \\ 0 & -1/|\hat{e}_4| & 0 & 1/|\hat{e}_4| \\ 0 & -1/|\hat{e}_5| & 0 & 0 \\ 0 & 0 & 1/|\hat{e}_6| & -1/|\hat{e}_6| \\ 0 & 0 & 1/|\hat{e}_7| & 0 \\ 0 & 0 & 0 & -1/|\hat{e}_8| \end{bmatrix}. \tag{36b}$$

4.7 Interpolation

One motivation for having interpolating functions is the convective term in the Navier-Stokes equations (2), namely $\boldsymbol{u} \cdot \nabla \boldsymbol{u}$. Here, when discretising, we note that the gradient is cell-to-face, and by evaluating the discrete convection (30) at $\boldsymbol{u}_c$, we get

$$[C(\boldsymbol{u}_f)\boldsymbol{u}_c]_K. \tag{37}$$

This term represents the convection of $\boldsymbol{u}$ on cell K. From this, we notice that both a cell-centred and a face-centred velocity are needed. Another motivation for the interpolation arises when dealing with the time marching procedure and the Poisson pressure problem, as detailed in Section 5.

We then define two interpolations: cell-to-face and face-to-cell. These functions are respectively denoted by $\Gamma_{c\to f}$ and $\Gamma_{f\to c}$, where they are not quite the inverses of each other; $\Gamma_{c\to f}\Gamma_{f\to c} \approx I$. This approximation instead of equality is due to the boundary conditions, which appear in and affect the interpolations. The boundary conditions appearing in the interpolation functions is highlighted in Section 4.8.

The interpolations defined here are similar to the ones defined by Trias *et al.* [19], but they differ when comparing the specific scalings chosen.

4.7.1 Face-to-cell interpolation

The face-to-cell interpolation is defined by a weighted average of the values on the faces around a cell. Given a face-centred velocity $\boldsymbol{u}_c \in \mathbb{R}^{n_f}$, we define its interpolation on cell K as

$$(\boldsymbol{u}_c)_K = [\Gamma_{f\to c}(\boldsymbol{u}_f)]_K := \frac{\sum_{e\in\partial K}(\boldsymbol{u}_f)_e\,|e|\,\boldsymbol{n}_e}{\sum_{e\in\partial K}|e|}. \tag{38}$$

4.7.2 Cell-to-face interpolation

The cell-to-face interpolation is defined by a weighted average of the values on the cells on either side of a face. Give a cell-centred velocity $\boldsymbol{u} \in \mathbb{R}^{n_c\times d}$, its interpolation on face $e = K|L$ between cells K, L is defined by

$$(\boldsymbol{u}_f)_e = [\Gamma_{c\to f}(\boldsymbol{u}_c)]_e := \frac{(\boldsymbol{u}_c)_K\,|K| + (\boldsymbol{u}_c)_L\,|L|}{|K| + |L|}\cdot\boldsymbol{n}_e, \quad e = K|L, \tag{39}$$

where again, for the boundary faces, we proceed by considering a ghost mirrored cell. This interpolation contains the boundary conditions, as can be seen in the next section.

4.8 Boundary conditions

In this section we look in closer detail to the boundary conditions and their implementation, for the case of Neumann and Dirichlet boundary conditions. We suppose that the boundary conditions are applied to the velocity, but the same reasoning holds for the pressure, by substituting the appropriate terms.

4.8.1 Neumann boundary conditions

As was illustrated in the two-dimensional example in equation (34), the gradient of the velocity on a boundary face e is given by

$$\operatorname{grad} u_e = \frac{\boldsymbol{u}_{K'} - \boldsymbol{u}_K}{|\hat{e}|} n_{e,K} \in \mathbb{R}^d, \tag{40}$$

where $\boldsymbol{u}_K$ denotes the velocity at the cell-centre of cell K inside the domain and $\boldsymbol{u}_{K'}$ is the velocity at the ghost mirrored cell centre, as explained in Section 3.4. Then, if we have a zero Neumann boundary condition on this boundary, this means that

$$\boldsymbol{u}_{K'} = \boldsymbol{u}_K, \tag{41}$$

or in other words, the cell-to-face interpolation on the boundary face must be

$$(\boldsymbol{u}_f)_{e\in\Gamma_N} = [\Gamma_{c\to f}(\boldsymbol{u}_c)]_{e\in\Gamma_N} := \frac{1}{2}(\boldsymbol{u}_c)_K\cdot\boldsymbol{n}_e, \tag{42}$$

where Γ_N denotes the part of the domain boundary $\partial\Omega$ on which Neumann boundary conditions are applied.

For a nonzero Neumann boundary condition, say $\frac{\partial \boldsymbol{u}}{\partial \boldsymbol{n}} = \alpha$, the ghost value is

$$\boldsymbol{u}_{K'} = |\hat{e}|\,n_{e,K}\alpha\boldsymbol{u}_K, \tag{43}$$

so the cell-to-face interpolation on the boundary is then

$$(\boldsymbol{u}_f)_{e\in\Gamma_N} = \frac{1}{2}(1 + |\hat{e}|\,n_{e,K}\alpha)\boldsymbol{u}_K\cdot\boldsymbol{n}_e. \tag{44}$$

4.8.2 Dirichlet boundary conditions

Dirichlet boundary conditions are also applied on the cell-to-face interpolation, in a very straightforward way. Suppose that on $\Gamma_D \subset \partial\Omega$ we enforce the boundary conditions $u_e = \alpha$, this is translated in the interpolation as

$$(\boldsymbol{u}_f)_{e\in\Gamma_D} = [\Gamma_{f\to c}(\boldsymbol{u}_c)]_{e\in\Gamma_D} := \alpha. \tag{45}$$

5 Time marching

In this section we aim to derive a time marching scheme for the Navier-Stokes equations (2). This is the natural next step after deriving a spatial discretisation scheme. To this end, we use an Explicit Euler discretisation, given by

$$\frac{\partial \boldsymbol{u}}{\partial t} \approx \frac{\boldsymbol{u}^{n+1} - \boldsymbol{u}^n}{dt} = f(\boldsymbol{u}^n, t_n), \tag{46}$$

where the superscript/subscript n denotes the timestep at which we are, $n+1$ is then the next step, and the stepsize is given by dt. The right hand side of (46) represents the spatial discretisation at timestep n. We choose this method for its simplicity to implement, but we then must be careful with stability requirements since this is a conditionally stable method. Applying the explicit Euler method to the incompressible Navier-Stokes equations yields

$$\nabla \cdot \boldsymbol{u}^n = 0 \tag{47a}$$

$$\frac{\boldsymbol{u}^{n+1} - \boldsymbol{u}^n}{dt} = -\boldsymbol{u}^n \cdot \nabla \boldsymbol{u}^n - \nabla p^n + \frac{1}{\mathrm{Re}} \nabla^2 \boldsymbol{u}^n. \tag{47b}$$

The second equation for the conservation of momentum can be split into two by introducing an intermediate velocity $\boldsymbol{u}^*$ such that

$$\frac{\boldsymbol{u}^{n+1} - \boldsymbol{u}^*}{dt} + \frac{\boldsymbol{u}^* - \boldsymbol{u}^n}{dt} = \boldsymbol{u}^n \cdot \nabla \boldsymbol{u}^n - \nabla p^n + \frac{1}{\mathrm{Re}} \nabla^2 \boldsymbol{u}^n. \tag{48}$$

The splitting wis then as follows. The first equation to solve is the convection-diffusion equations with all terms at timestep n. The second equation contains a terms at timestep $n+1$ on the left hand side and the pressure gradient on the right hand side. These equations are solved sequentially. This then gives a Poisson problem for the pressure.

5.1 Pressure Poisson problem

5.1.1 Continuous

In a spatially continuous setting, the velocity and pressure are solved in three consecutive steps.

1. We compute an intermediate velocity

$$\boldsymbol{u}^* = \boldsymbol{u}^n - dt \boldsymbol{u}^n \cdot \nabla \boldsymbol{u}^n + \frac{dt}{\mathrm{Re}} \nabla^2 \boldsymbol{u}^n. \tag{49}$$

   This is the first equation mentioned above, consisting of the convection-diffusion equation.

2. We consider the remaining terms from the time-discrete incompressible Navier-Stokes equations, and we take the divergence:

$$\implies \nabla^2 p^n = \frac{1}{dt} \nabla \cdot \boldsymbol{u}^*. \tag{50}$$

   This follows from the fact that the flow is assumed incompressible at all timesteps n.

3. The velocity is updated from the intermediate velocity and the pressure found from solving the Poisson problem for p^n (50)

$$\boldsymbol{u}^{n+1} = \boldsymbol{u}^* - dt \nabla p^n. \tag{51}$$

5.1.2 Discrete

In a spatially discrete setting, the velocity and pressure are updated in a similar way, where now all equations are discretised.

1. Compute an intermediate velocity

$$\boldsymbol{u}_c^* = \boldsymbol{u}_c^n - dt C(\boldsymbol{u}_f^n) \boldsymbol{u}_c^n + \frac{dt}{\mathrm{Re}} L \boldsymbol{u}_c^n, \tag{52}$$

   where here we have a cell-centred intermediate velocity since both the convection and the diffusion return cell-centred values.

2. Solve a Poisson problem for the pressure

$$L\boldsymbol{p}_c = \frac{1}{dt} D\boldsymbol{u}_f^*, \tag{53}$$

where we then need to apply a cell-to-face interpolation to the intermediate velocity $\boldsymbol{u}_c^*$. Since the Laplacian is nonsingular (see Section 4.4), the Poisson problem results in a unique solution.

3. Update the velocity

$$\boldsymbol{u}_f^{n+1} = \boldsymbol{u}_f^* - dt\nabla \boldsymbol{p}_c^n. \tag{54}$$

Then, another face-to-cell interpolation must be applied to recover $\boldsymbol{u}_c^{n+1}$ and start again from step 1.

6 Numerical implementation

We then aim to implement the spatial and temporal discretisations derived respectively in Sections 3, 4 and 5. This section follows the same structure as the global structure of the code, as illustrated in Figure 9.

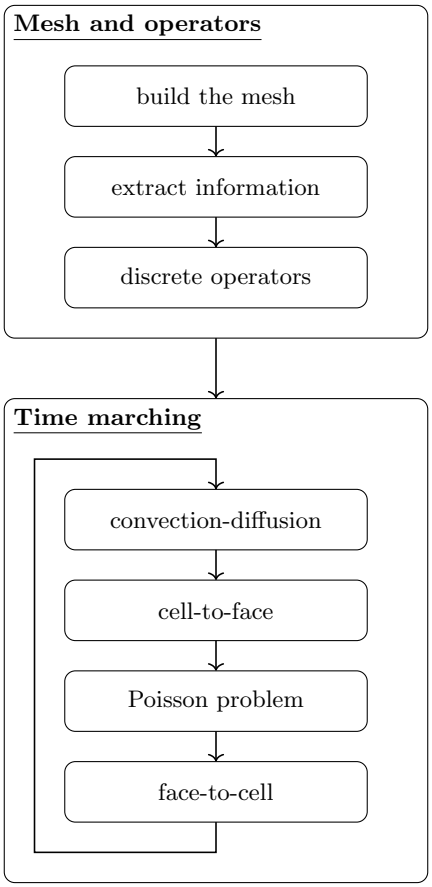


Figure 9. The global structure of the code. The following sections explain each block in further detail. The blocks cell-to-face and face-to-cell in the time marching refer to the respective interpolations.

6.1 Julia code

The full code can be found at https://github.com/MileneMdvl/MasterThesis. In this section, we aim to provide a brief explanation of the different files, packages, and functions used. We proceed in the same structure and order as Figure 9, which details the global structure of the code. The language Julia was chosen for it's relatively simple structure and syntax, and for its increased computational speed compared to e.g. Python.

6.1.1 Building the mesh

To obtain a Delaunay triangulation from a set of points, we use the package `Delaunay` [16]. This package was chosen over others due to it's ability to triangulate and tetrahedralise both two- and three-dimensional domains. Some further work could be done to test the different packages available, such as [6, 18, 20], and compare them with regards to e.g. accuracy, runtime, ability to implement mesh refinement, etc. We implemented two options for the set of points.

- Randomly generated points on $\Omega = [0,1] \times [0,1]$. These points are uniformly distributed, where we also added the criteria that, given N generated points, no two points can be within $1/N$ distance of one another. This was done with the aim to avoid invalid triangulations. There is also the possibility to generate uniformly distributed points on the three-dimensional cube $\Omega = [0,1] \times [0,1] \times [0,1]$.
- Regular meshing of points on $\Omega = [0,1] \times [0, \frac{\sqrt{3}}{2}]$, as in Section 3.5.

Then, given the points, we generate the Delaunay triangulation, which outputs the points, mesh, and lists of vertices, faces, boundary faces, and cells. The former two are returned to be able to plot the triangulation. The latter four lists are then used as inputs for the discrete operators. The lists of faces and cells are given in terms of the indices corresponding to the vertices connecting them.

In order to manipulate these lists easily on Julia, they are actually returned as vectors of vectors. The same principle holds for a three-dimensional domain, where each vertex has three components, each face has three vertices (i.e. three components) and each cell has four vertices (i.e. four components).

For instance, recalling the labelling of the discretisation in Figure 8, the list of vertices is an ordered list of arrays of the coordinates of each vertex, where the first entry corresponds to vertex labelled 1, and so on. The list of faces for this example is

$$[[1,3],[3,2],[2,1],[4,1],[2,4],[1,5],[5,3],[4,5]]. \tag{55}$$

Note that this each face appears once, but the order in which the face appears (i.e. $[i,j]$ instead of $[j,i]$) is not important for any following operations. The list of boundary faces is a subset of the face list, i.e. in this example,

$$[[3,2],[5,3],[2,4],[4,5]], \tag{56}$$

where again each face appears once and the order is the same as in the list of faces. Finally, the list of cells is

$$[[1,3,2],[4,1,2],[1,5,3],[5,1,4]], \tag{57}$$

where each cell also appears once.

Generating the points and the triangulations is given in the file `build_mesh.jl`. As of now, the ability to generate a regular mesh is only possible for a two-dimensional domain. The three-dimensional case should follow the same reasoning, and the extension would be rather straightforward. The function to tetrahedralise was not tested beyond the generation of the mesh and the duality condition test in Section 7.1

We also make use of dictionaries to store local information about these cells and faces. Namely, we have a dictionary that stores per vertex i which cells contain this vertex, and it only stores the indices of the cells within the list. Another dictionary stores per cell index i which faces lie on its boundary, and once again only the indices of the faces are stored. These dictionaries are then used when computing the gradient, divergence and interpolations, as we then avoid having a loop over all cells and all faces.

6.1.2 Extracting information from the mesh

Once the mesh is built, we need to extract certain information from it, in order to then implement the discrete operators. For instance, we need a functions that given either a cell or a face, returns the corresponding volume, either $|K|$ or $|e|$. This function needs to be general, in the sense that cells can either be three- or four-dimensional vectors, and faces either two- or three-dimensional. Similarly, the circumcentre needs to be computed, for triangles, tetrahedra, and lines (where the former is used when computing the dual edge $|\hat{e}|$ for boundary faces).

We also implemented a function which, given a cell K, returns all of its faces $e \in \partial K$. Similarly, given a face e and a cell K, another functions determines whether $e \in \partial K$ or not. In the same idea as the aforementioned function, given a face e we can determine the two cells K and L adjacent to this face, with a special treatment for boundary faces. These functions are crucial to the discrete operators, since for instance when computing the divergence we sum over the faces bordering a given cell, and when computing the gradient on a face we require the two neighbouring cells. These aforementioned functions can be found in the file `mesh_functions.jl`.

6.1.3 Building the discrete operators

To build the discrete divergence and gradient, we define sparse matrices $D \in \mathbb{R}^{n_c \times n_f}$ and $G \in \mathbb{R}^{n_f \times n_c}$ as in equations (27) and (28). The Laplacian is then the product of these matrices, and applying e.g. the gradient to the pressure $\boldsymbol{p}_c$ is simply a matrix-vector multiplication. The convection is computed as in (30), using the gradient operator. The discrete divergence, gradient and convection are given in the file `divgrad.jl`.

6.1.4 Implementing the time marching

The time marching is implemented as described in Section 5.1.2. At the first iterate, compute the cell-centred intermediate velocity $\boldsymbol{u}_c^*$ from the convection-diffusion equation, using the convection and Laplacian operators from Section 6.1.3. We then apply cell-to-face interpolation to obtain the face-centred intermediate velocity $\boldsymbol{u}_f^*$. Using the Laplacian once again, we solve a Poisson problem for the pressure, giving the cell-centred pressure $\boldsymbol{p}_c^n$. Taking it's gradient yields the face-centred velocity at the second iterate $\boldsymbol{u}_f^{n+1}$. Applying face-to-cell interpolation gives the cell-centred velocity at the second time step. We can then compute the intermediate velocity and repeat the same process for all time steps. This time marching is presently in the file `2d_mesh_test.jl`, as it was used for the testing of the code as explained in Section 7.

7 Code verification

Code verification is an important tool in numerical mathematics as it allows to make sure that there are no mistakes in the implementation or 'bugs' in the code. We also aim to verify that the numerical solution converges with the right order, in space. We do this in two steps. First, we check that the duality condition with which the gradient was derived does hold. Next, we implement the method of manufactured solution for a more in-depth check of the code and of the time marching.

7.1 Checking the duality condition

The first step of code verification we apply is checking whether the duality condition between the gradient and the divergence derived in Section 4.1 holds, that is whether

$$\left\langle \boldsymbol{u}_f, \operatorname{grad} \boldsymbol{p}_f \right\rangle_{\mathcal{F}} = - \left\langle \operatorname{div} \boldsymbol{u}_c, \boldsymbol{p}_c \right\rangle_{\mathcal{C}} . \tag{58}$$

The gradient was defined from this relation, and hence, we aim to verify that not mistakes are present in the gradient and divergence. Note that this is not a complete test, the proper analysis is done in Section 7.2, when applying the method of manufactured solutions. This duality condition test should be seen more as a sanity check than anything else.

We run the simulation for various values of N, both in two- and three-dimensions, for randomly generated points. We also use randomly generated (also uniformly distributed) values for the cell-centred pressure and face-centred velocity. Finally, we plot the sum of the two discrete inner products, and we check that this indeed returns zero. The plots can be found in Figure 10, for both two- and three-dimensional settings. Note that for each value of N, we run the simulation (that is, we build the mesh and compute the operators and inner products) ten times, and we plot this average.

From this, we can clearly see that indeed the duality condition is satisfied. This then leads us to a more detailed and complete code verification technique.

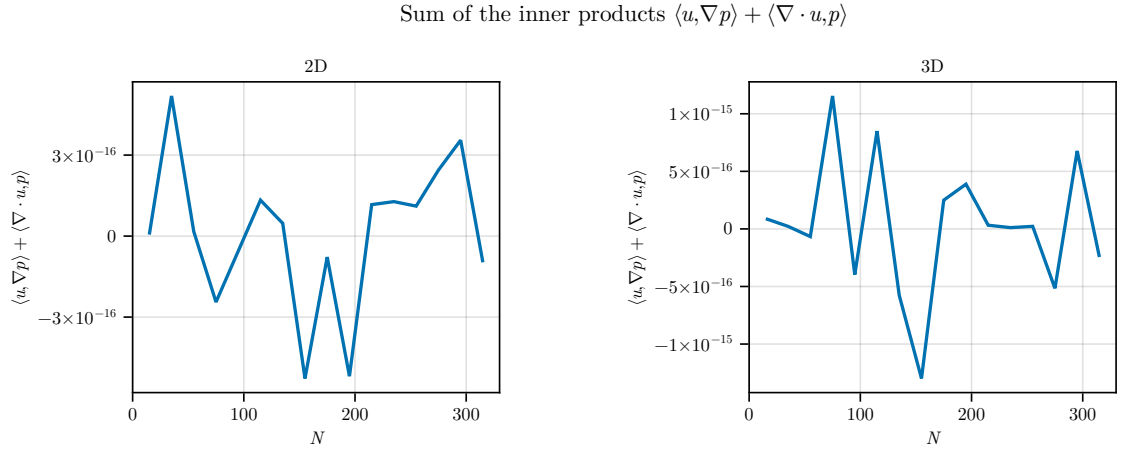


Figure 10. Plot of the sum of the discrete inner products $\langle \boldsymbol{u}_f, \operatorname{grad} \boldsymbol{p}_f \rangle_{\mathcal{F}} + \langle \operatorname{div} \boldsymbol{u}_c, \boldsymbol{p}_c \rangle_{\mathcal{C}}$, to check that the duality condition is indeed satisfied, for the two- (left) and three-dimensional (right) cases.

7.2 Method of Manufactured Solutions

7.2.1 The method

The method of manufactured solutions (MMS) is a code verification technique. As the name suggests, we use a pre-chosen (manufactured) solution, and we check how the code behaves and whether the given solution is recovered [14]. Since our implementation is a dicretisation of the Navier-Stokes equations, our chosen solution will not be exact. Hence, a source term is obtained from the steady-state equations of motion

$$\boldsymbol{f} = \boldsymbol{u}_0 \cdot \nabla \boldsymbol{u}_0 + \nabla p_0 - \frac{1}{\mathrm{Re}} \nabla^2 \boldsymbol{u}_0, \tag{59}$$

where the subscripts $\boldsymbol{u}_0, p_0$ are used to denote the manufactured velocity and pressure solutions. Steady-state means that the time derivative is set to zero. Then, this source term appears in the first step of the time marching, when computing the intermediate velocity (52), so we now have

$$\boldsymbol{u}_c^* = \boldsymbol{u}_c^n - dt C(\boldsymbol{u}_f^n)\boldsymbol{u}_c^n + \frac{dt}{\mathrm{Re}} L \boldsymbol{u}_c^n - dt \boldsymbol{f}_c, \tag{60}$$

where the source term is then to be taken at cell centres. After applying time marching, the goal is to recover the manufactured solutions $\boldsymbol{u}_0, p_0$.

7.2.2 Choice of manufactured solution

There are many choices of manufactured solutions possible – in fact, any choice would work, as long as we are able to satisfy certain conditions. These conditions are the following. The flow must be divergence free, since the discretisation that we aim to verify holds for incompressible flows. Secondly, the manufactured solutions must satisfy the desired boundary conditions.

In our case, we consider the regular triangulation of the two-dimensional domain $\Omega = [0,1] \times [0, \frac{\sqrt{3}}{2}]$, as detailed in Section 3.5. We also consider Neumann boundary conditions, implemented within the cell-to-face interpolation (see Section 4.8). We make the following choice of manufactured solutions

$$u_0(x,y) = \frac{2}{\sqrt{3}} \cos(2\pi x) \sin\left(\frac{4\pi}{\sqrt{3}} y\right), \tag{61a}$$

$$v_0(x,y) = -\sin(2\pi x) \cos\left(\frac{4\pi}{\sqrt{3}} y\right), \tag{61b}$$

$$p_0(x,y) = \cos(2\pi x) \cos\left(\frac{4\pi}{\sqrt{3}} y\right). \tag{61c}$$

This yields a divergence free solution, since

$$\begin{aligned}\nabla \cdot \boldsymbol{u} &= \frac{\partial u_0}{\partial x} + \frac{\partial v_0}{\partial y} \\ &= \frac{2}{\sqrt{3}} \cdot 2\pi \sin(2\pi x) \sin\left(\frac{4\pi}{\sqrt{3}} y\right) - \sin(2\pi x)\left(-\frac{4\pi}{\sqrt{3}}\right) \sin\left(\frac{4\pi}{\sqrt{3}} y\right) \\ &= 0.\end{aligned} \tag{62}$$

Moreover, the velocity also satisfies the Neumann boundary condition $\frac{\partial \boldsymbol{u}}{\partial \boldsymbol{n}} = 0$. We can split the boundary of the domain into $\Gamma_1, \Gamma_2, \Gamma_3, \Gamma_4$, ordered clockwise starting at the left side, with respective outward normal vectors $\boldsymbol{n}_i$, for $i = 1, 2, 3, 4$, as illustrated in Figure 11. Then, looking at the normal derivatives, we only treat boundaries Γ_1, Γ_2, where boundaries Γ_3, Γ_4 follow the same logic.

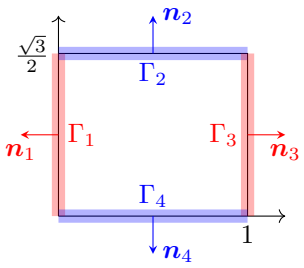


Figure 11. Sketch of the domain $\Omega = [0, 1] \times [0, \frac{\sqrt{3}}{2}]$ with the boundary split into 4 sections, each with their respective outward normal vectors.

The Jacobian of the velocity if given by:

$$J_{\boldsymbol{u}_0} = \begin{bmatrix} \frac{\partial u_0}{\partial x} & \frac{\partial u_0}{\partial y} \\ \frac{\partial v_0}{\partial x} & \frac{\partial v_0}{\partial y} \end{bmatrix} = \begin{bmatrix} -\frac{4\pi}{\sqrt{3}} \sin(2\pi x) \sin(\frac{4\pi}{\sqrt{3}} y) & \frac{8\pi}{3} \cos(2\pi x) \cos(\frac{4\pi}{\sqrt{3}} y) \\ -2\pi \cos(2\pi x) \cos(\frac{4\pi}{\sqrt{3}} y) & \frac{4\pi}{\sqrt{3}} \sin(2\pi x) \sin(\frac{4\pi}{\sqrt{3}} y) \end{bmatrix}. \tag{63}$$

The derivative of u_0 in the direction of $\boldsymbol{n}_1$ is then given by:

$$\begin{bmatrix} -\frac{4\pi}{\sqrt{3}} \sin(2\pi x) \sin(\frac{4\pi}{\sqrt{3}} y) & \frac{8\pi}{3} \cos(2\pi x) \cos(\frac{4\pi}{\sqrt{3}} y) \end{bmatrix} \begin{bmatrix} -1 \\ 0 \end{bmatrix} = \frac{4\pi}{\sqrt{3}} \sin(2\pi x) \sin(\frac{4\pi}{\sqrt{3}} y). \tag{64}$$

And on Γ_1, $x = 0$, so this yields

$$\left.\frac{\partial u_0}{\partial \boldsymbol{n}_1}\right|_{\Gamma_1} = 0. \tag{65}$$

Similarly, for the normal derivative of u_0 in direction $\boldsymbol{n}_3$ at Γ_3, we find $\left.\frac{\partial u_0}{\partial \boldsymbol{n}_3}\right|_{\Gamma_3} = 0$, since on this boundary it holds that $x = 1$. For the y-component, we have the normal derivative of v_0 in direction $\boldsymbol{n}_2$ given by

$$\begin{bmatrix} -2\pi \cos(2\pi x) \cos(\frac{4\pi}{\sqrt{3}} y) & \frac{4\pi}{\sqrt{3}} \sin(2\pi x) \sin(\frac{4\pi}{\sqrt{3}} y) \end{bmatrix} \begin{bmatrix} 0 \\ 1 \end{bmatrix} = \frac{4\pi}{\sqrt{3}} \sin(2\pi x) \sin(\frac{4\pi}{\sqrt{3}} y), \tag{66}$$

and on Γ_2, $y = \frac{\sqrt{3}}{2}$, and so

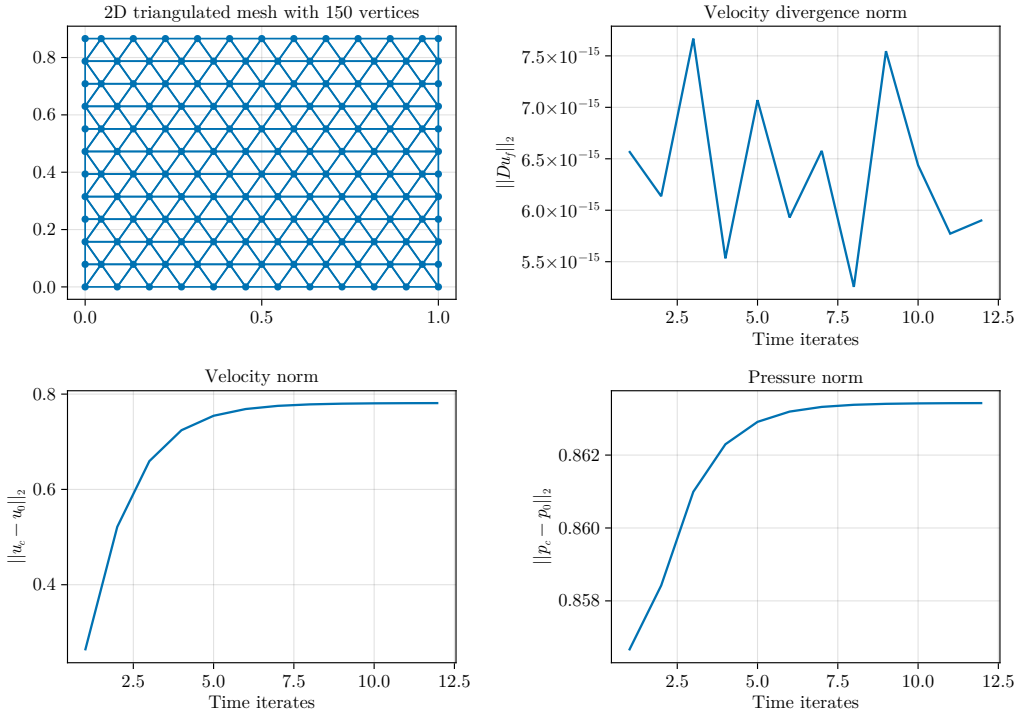
$$\left.\frac{\partial v_0}{\partial \boldsymbol{n}_2}\right|_{\Gamma_2} = 0. \tag{67}$$

The same holds on Γ_4. Therefore, the velocity field chosen satisfies the zero Neumann boundary condition.

7.2.3 Results

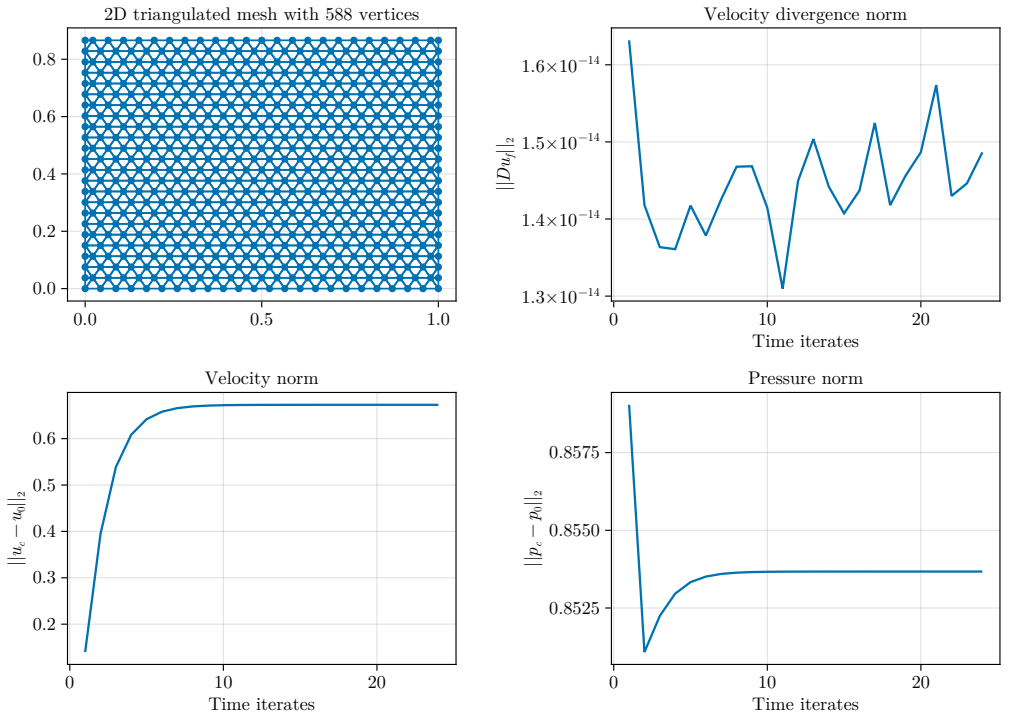
We then run the code for $N = 6, 12, 24, 48, 96, 192, 384$, where N determines how many vertices lie on each side of the domain. The outputs are given for all values in Appendix D.1, and in Figure 12 for $N = 12, 24$. We only display these results here as they are sufficiently representative of the behaviour of the solutions as the number

$Re = 1600,\ N = 12,\ dt = 0.08333,\ dx = 0.08333$



(a) $N = 12,\ \|\boldsymbol{u} - \boldsymbol{u}_0\| \to 0.78111,\ \|p - p_0\| \to 0.86343.$

$Re = 1600,\ N = 24,\ dt = 0.04167,\ dx = 0.04167$



(b) $N = 24,\ \|\boldsymbol{u} - \boldsymbol{u}_0\| \to 0.67291,\ \|p - p_0\| \to 0.85367.$

Figure 12. Four plots highlighting the solution of the time marching, for different values of N. In both subplots, the top left plot shows the regular mesh discretisation. The top right plot shows the evolution of the norm of the divergence of the velocity field over time, taken at face centres and normalised by the number of faces. The bottom left plot shows the norm difference $\|\boldsymbol{u} - \boldsymbol{u}_0\|$ taken at cell centres and normalised by the number of cells. The bottom right plot shows the norm difference $\|p - p_0\|$ also taken at cell centres and also normalised. The value to which these norms converge are indicated in the caption of each subfigure.

of vertices increases. All plots are included in this report for completeness of the analysis. Note that when we specify that the norms are normalised by the number of cells, we mean that e.g. the pressure norm is

$$\|p - p_0\| := \frac{\|\boldsymbol{p}_c - p_0\|_2}{\sqrt{n_c}}, \tag{68}$$

where $\|\cdot\|_2$ denotes the standard Euclidean norm, n_c is the number of cells, and $\boldsymbol{p}_c$ is the cell-centred pressure. The velocity norm is obtained similarly.

We set $dt = dx$, where $dx = 1/N$, as the velocity is less than one in this case, and this was furthermore found to be a sufficient setting. However, when running the code for $N = 384$, this lead to a diverging solution, and hence we set $dt = dx/2$ in that case.

In all of these figures, we can note that the divergence of the velocity field is always zero, where its order for the different values of N is highlighted in Table 3. We can also note that the solution obtained from the

N	6	12	24	48	96	192	384
$\|D\boldsymbol{u}_f\|$	$\sim 10^{-15}$	$\sim 10^{-15}$	$\sim 10^{-14}$	$\sim 10^{-14}$	$\sim 10^{-14}$	$\sim 10^{-13}$	$\sim 10^{-13}$

Table 3. Order of $\|D\boldsymbol{u}_f\|$ for different values of N. This order can be read from the top right plot in Figure 12 and in Appendix D.1.

simulation and the time marching procedure is not the same as the initial manufactured solution. Indeed, and as can be seen in Figure 13, despite having a convergence behaviour in space and time, the error between the two solutions does not converge to zero, neither for the velocity nor the pressure. One explanation for this could be the choice of manufactured solutions itself, which would introduce numerical errors for the approximation of the sines of cosines. There could also be some underlying error in the numerical implementation, resulting in the correct order of convergence (as shown in Section 7.2.4), yet yielding an incorrect solution altogether.

7.2.4 Order of convergence

From these figures, we also note that the norm difference between the velocity and pressure at the end of the time marching and the initial velocity and pressure converges over time. Indeed, as the time marching goes on, these norms stabilise. Furthermore, looking carefully at their limits, we see that it decreases as N increases (doubles). Note that in the caption of the figures we truncated the value. These limits are given in Table 4 for different N. These limits are also plotted in Figure 13, for a more visual interpretation.

N	$\lim \|\boldsymbol{u} - \boldsymbol{u}_0\|$	$\lim \|p - p_0\|$
6	1.01654092612348	0.8646155109644296
12	0.7811112243762136	0.8634258303912086
24	0.6729116554791046	0.853672656748472
48	0.6337338216574162	0.8479366184438532
96	0.6204211216049622	0.8450020541824618
192	0.6156883365866228	0.8435333562542691
384	0.6136617195627028	0.8419934165870117

Table 4. Limit as the number of iterates increases of the velocity and pressure difference with regards to the manufactured solutions for different values of N.

It can then be seen that there is some convergence happening when N is increased. We then aim to find the order of convergence. To this end, we turn to computing the q-ratio. Given an approximation $\tilde{f}_N$, corresponding to N vertices, we can estimate the order of convergence in space by

$$q := \left| \frac{\tilde{f}_N - \tilde{f}_{2N}}{\tilde{f}_{2N} - \tilde{f}_{4N}} \right|. \tag{69}$$

We then say that $\tilde{f}$ converges to some f^* with order $\mathcal{O}(1/N^p)$, where p is such that $q \approx 2^p$ [10].

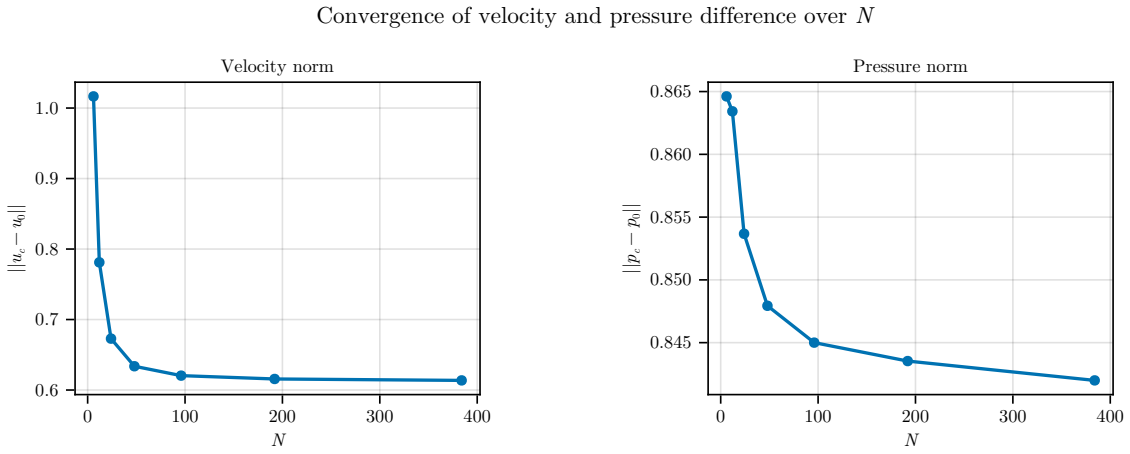


Figure 13. Plot of the values of Table 4, showcasing the convergence behaviour of the norms. Here again, both norms are taken at cell-centres and normalised by the number of cells.

Theoretically, we expect the method to be 1^{st} order in space [19]. Namely, this can be seen by recalling the gradient (25), and applying it to a regular rectangular grid (Figure 3), then we recover a first order structured finite difference discretisation of the gradient [11]. Therefore, we expect $p = 1$, or in other words, we expect the q-ratio to be about 2.

Figure 14 presents the plots of the q-ratios for the velocity and the pressure, computed at $N = 6, 12, 24, 48, 96$. Note that we do not compute the q-ratios for higher values of N, since it requires a grid refinement $4N$, and therefore if we take e.g. $N = 192$ we would require $N = 768$ points in each direction, which makes a total of approximately $3,589,824$ vertices. This number is slightly too big for the computations, as can be further seen from looking at the runtime in Section 7.2.5.

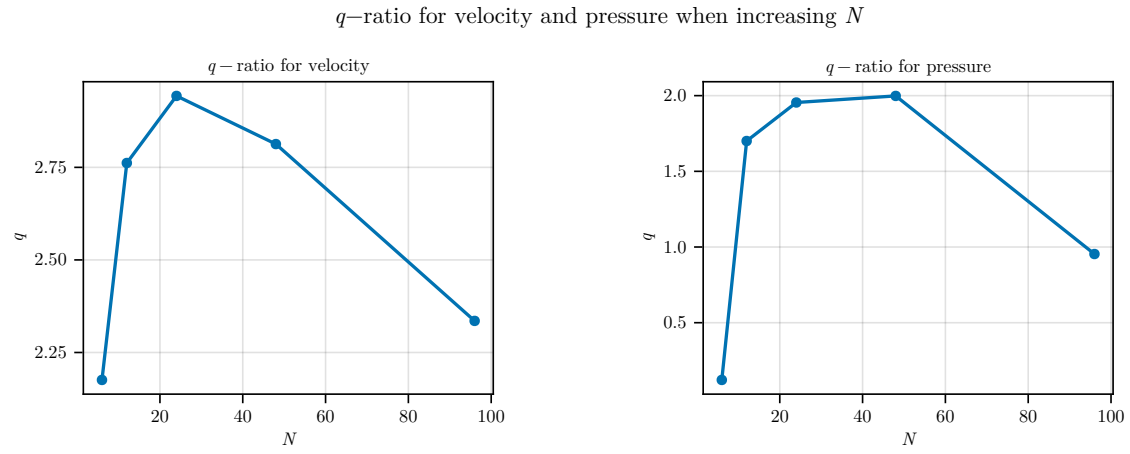


Figure 14. The q-ratios (69) computed for the velocity difference $\|\boldsymbol{u} - \boldsymbol{u}_0\|$ (left) and the pressure difference $\|p - p_0\|$ (right) for $N = 6, 12, 24, 48$, where the corresponding values for the computations are given in Table 4.

From Figure 14, we can see that the q-ratio for the velocity is about 2, where we can see that it does increase at first, but then approaches 2 again. For the pressure however, the q-ratio approaches 2 at first, but then decreases again, to about 0.9. This could be explained from the fact that the method is first order, hence the cell-centred velocity $\boldsymbol{u}_c$ converges with order $\mathcal{O}(1/N^2)$ (or equivalently, $q = 2$). From this, we apply interpolation to obtain the face-centred velocity $\boldsymbol{u}_f$, which is then less accurate. Hence, this also makes the pressure $\boldsymbol{p}_c$ less accurate, since it relies on the divergence of the face-centred velocity. Which is why we see a higher q-ratio for the velocity than for the pressure.

7.2.5 Runtime

We compute the runtime for the two main elements of the code, namely generating the mesh and computing the time marching. For smaller N values (i.e. up until 192) we run the simulation five times and take the average time. For $N = 384$, the simulation took too long for the averaging to be feasible.

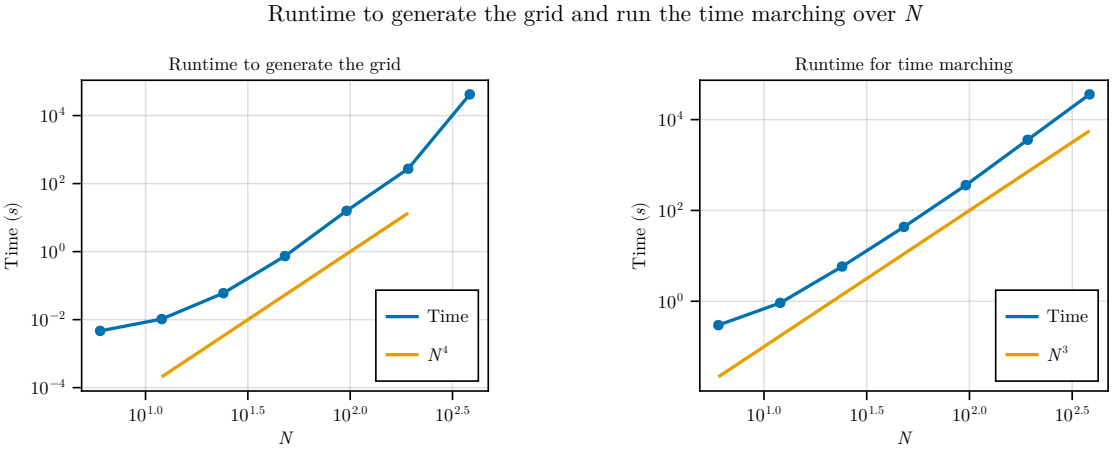


Figure 15. Total runtime in seconds to generate a regular grid (left) and to implement the time marching (right). The grid generation scales as N^4 while the time marching scales as N^3.

For $N = 384$, the mesh generation took approximately 1 hour and 15 minutes and the time marching a little over 10 hours. The time marching for this N was also run on a different (more powerful) computer, where it took about 7 hours. The times indicated in Figure 15 are all from the same machine for consistency, but a clear improvement can be seen by increasing the memory power.

8 Conclusion and discussion

In this paper we present a first-order finite-volume unstructured discretisation scheme for incompressible Navier-Stokes turbulence. We first introduced the main idea behind Large Eddy Simulation – namely filtering, which is done implicitly via the grid discretisation. We then looked at a specific grid obtain by Delaunay triangulation. This triangulation allows for a natural definition of primal and dual grids.

The divergence is discretised as a face-to-cell operator, where the divergence of the velocity on a cell is defined as the sum of the face-centred velocity on the faces bordering the cell. The discrete gradient is minus the adjoint of the divergence, which results in a cell-to-face operator. The gradient on a face is the difference of the cell-centred values, for the cells adjacent to a face. These operators give straightforward definitions for the convection and diffusion operators, relating them to their continuous definitions. Face-to-cell and cell-to-face interpolations are also introduced, where the latter also contain the Dirichlet and Neumann boundary conditions. The form of the discretisation derived in the paper appears to be somewhere in between [8] and [19]. We present an explicit way to compute the divergence and gradient as done in the former, but we apply the convection and diffusion in a closer manner to the latter.

A time marching procedure is implemented, by considering an explicit Euler time-discretisation. We first compute an intermediate velocity from the convection-diffusion equation. Then, the pressure is updated by solving a Poisson problem. Finally, velocity is updated from the pressure.

The building of the mesh, of the operators, and the time marching procedure are implemented in Julia. We first show that with this implementation, it does hold that the gradient and the divergence are adjoints to one another. We then look into more detail at code verification techniques, and particularly at the method of manufactured solutions. This method consists in choosing an initial solution for the pressure and velocity, and substituting it in the steady-state form of the Navier-Stokes equations. As this is not an exact solution,

we obtain a source term, which must then be added to the convection-diffusion equation in the time marching. The aim of this method is then to recover the manufactured solutions at the end of the time iterates.

We were not able to recover these solutions exactly. In particular, we obtain solutions for which the difference in norm with regards to the initial manufactured solutions converges to about 0.61 for the cell-centred velocity and about 0.84 for the cell-centred pressure. This large error may be explained partly from the choice of manufactured solutions made. Namely, we chose a combination of sines and cosines, for which there will be some numerical errors arising. However, the error we found seems too significant for this to be the only reason. Hence, we suppose that there is some error present in the numerical implementation. Some further research could be done to determine where these errors specifically lie. A first guess could be to check the accuracy of the interpolating functions, and perhaps to compare them with another choice of scaling, as in e.g. [19].

When considering the result of the method of manufactured solutions, we notice that we obtain the correct order of convergence for the velocity, namely 1^{st} order. The pressure has a lower order of convergence, which can be explained by the fact that the pressure is obtained from the face-centred velocity, and by applying interpolation, this is less accurate that the cell-centred velocity.

Some further work could be then done in improving both the accuracy and the order of convergence in the numerical implementation. Namely, both could be improved upon by considering a different choice of interpolation, with a different weighting average of the cell- and face-centred values. Other choices of manufactured solutions could be checked, where the outcome should be the same regardless. That it, theoretically, regardless of the choice of manufactured solution made, we should recover this solution at the end of the time marching. The solutions should only satisfy the desired boundary conditions and yield a divergence-free velocity field.

Moreover, in the numerical implementation, we have only considered zero Neumann boundary conditions on the velocity. Further work could be done to verify the implementation of Dirichlet or even nonzero Neumann boundary conditions. Robin or periodic boundary conditions could also be added to the cell-to-face interpolation. This way, the numerical scheme would allow from a wider range of applications.

Further work could also be done on optimising the numerical implementation. Particularly with the aim of reducing the runtime of both the mesh generation and of the time marching. To this end, for the mesh generation, some comparison between the pre-existing packages that allow Delaunay triangulation could be done. This comparison could also be done with regards to other criteria, such as ease of implementation, extension to three-dimensional domains, mesh refinement. For instnace, the package we used did not allow for a straightforward and built-in mesh refinement procedure, so we had to build a regular mesh and do the refinement manually.

After verifying the numerical implementation of the method derived in this paper, further work could also be undertaken to make this model more accurate with regards to turbulence modelling. Namely, as mentioned briefly in Section 4.5, some regularisation can be applied to the convective term, to stop the production of small scale motions which arise from the nonlinear convection [19, 21].

Moreover, as stated in the introduction, in this paper we disregarded the residual stress tensor arising from the filtered Navier-Stokes equations. Modelling this term is a crucial step to Large Eddy Simulation, and hence further work could be done on implementing this. A first step could be to first look at the Smagorinsky model, which is further detailed in Appendix C.2. Then, improvements of this model could be considered, such as Vreman or (dynamic) Germano models [5, 22].

Acknowledgements

I would like to thank my parents for their continuous support and encouragement. I also thank my supervisor, Roel Verstappen, for his precious help in writing this thesis. Further thanks go to my friends who spent many hours listening to me talk enthusiastically about triangles. I also deeply thank Robin and Sietse for their invaluable help and support for the coding part of this project.

9 References

[1] J. Blazek. "Turbulence Modeling". *Computational Fluid Dynamics: Principles and Applications*. Elsevier, 2015, pp. 213–252. DOI: 10.1016/B978-0-08-099995-1.00007-5.

[2] R. N. Bracewell. *The Fourier transform and its applications*. 3rd ed. McGraw Hill, 2000.

[3] Y. S. Elshakhs, K. M. Deliparaschos, T. Charalambous, G. Oliva, and A. Zolotas. "A Comprehensive Survey on Delaunay Triangulation: Applications, Algorithms, and Implementations Over CPUs, GPUs, and FPGAs". *IEEE Access* 12 (2024). DOI: 10.1109/ACCESS.2024.3354709.

[4] M. Farrashkhalvat and J. P. Miles. "Basic Structured Grid Generation". Butterworth-Heinemann, 2003. DOI: 10.1016/B978-0-7506-5058-8.X5000-X.

[5] M. Germane, U. Piomelli, P. Moin, and W. H. Cabot. "A dynamic subgrid-scale eddy viscosity model". *Physics of Fluids* 3 (1991), pp. 1760–1765. DOI: 10.1063/1.857955.

[6] C. Geuzaine and J. .-. Remacle. "Gmsh: a three-dimensional finite element mesh generator with built-in pre- and post-processing facilities". *International Journal for Numerical Methods in Engineering* 79.11 (2009), pp. 1309–1331. DOI: 10.1002/nme.2579.

[7] A. N. Kolmogorov. "The Local Structure of Turbulence in Incompressible Viscous Fluid for Very Large Reynolds Numbers". *Proceedings: Mathematical and Physical Sciences* (1991).

[8] P. Korn and L. Linardakis. "A conservative discretization of the shallow-water equations on triangular grids". *Journal of Computational Physics* 375 (2018), pp. 871–900. DOI: 10.1016/j.jcp.2018.09.002.

[9] M. Lesieur. *Turbulence in Fluids*. In Fluid Mechanics and its Applications. Springer, 2008. DOI: 10.1007/978-1-4020-6435-7.

[10] R. Luppes. "Computational Methods of Science Part 1". Lecture Notes.

[11] P. J. Olver. *Introduction to Partial Differential Equations*. Springer, 2014. DOI: 0.1007/978-3-319-02099-0.

[12] S. B. Pope. *Turbulent Flows*. Cambridge University Press, 2000. DOI: 10.1017/CBO9780511840531.

[13] L. F. Richardson. *Weather Prediction by Numerical Processes*. Cambridge University Press, 1922.

[14] C. J. Roy, C. C. Nelson, T. M. Smith, and C. C. Ober. "Verification of Euler/Navier–Stokes codes using the method of manufactured solutions". *International Journal for Numerical Methods in Fluids* 44 (2004), pp. 599–620.

[15] P. Sagaut. *Large Eddy Simulation for Incompressible Flows. An Introduction*. 3rd ed. Springer, 2006. DOI: 10.1007/b137536.

[16] E. Schnetter. *Delaunay*. https://github.com/eschnett/Delaunay.jl. 2020.

[17] U. Schumann. "Subgrid scale model for finite difference simulations of turbulent flows in plane channels and annuli". *Journal of Computational Physics* 18 (1975), pp. 376–404. DOI: 10.1016/0021-9991(75)90093-5.

[18] K. Simon. *TriangleMesh*. https://github.com/konsim83/TriangleMesh.jl. 2020.

[19] F. X. Trias, O. Lehmkuhl, A. Oliva, C. D. Pérez-Segarra, and R. W. C. P. Verstappen. "Symmetry-preserving discretization of Navier–Stokes equations on collocated unstructured grids". *Journal of Computational Physics* 258 (2014), pp. 246–267. DOI: 10.1016/j.jcp.2013.10.031.

[20] D. J. VandenHeuvel. "DelaunayTriangulation.jl: A Julia package for Delaunay triangulations and Voronoi tessellations in the plane". *Journal of Open Source Software* 9.101 (Sept. 2024), p. 7174. DOI: 10.21105/joss.0717.

[21] R. Verstappen. "On restraining the production of small scales of motion in a turbulent channel flow". *Computers & Fluids* 37.7 (2008). DOI: 10.1016/j.compfluid.2007.01.013.

[22] A. W. Vreman. "An eddy-viscosity subgrid-scale model for turbulent shear flow: Algebraic theory and applications". *Physics of Fluids* 16.10 (2004), pp. 3670–3681. DOI: 10.1063/1.1785131.

A Reynolds equations

In this section we derive the Reynolds-Averaged Navier-Stokes equations, which are at the basis of the Reynolds stress models. These models solve the equations of motion of a turbulent flow by averaging the equations in time and then solving the averaged equations on a discrete grid. The averaging works as a damping for the turbulent motions, and this technique is referred to as Reynolds-Averaged Navier-Stokes (RANS). In terms of computational cost, this method is cheaper than both DNS (directly solving the Navier-Stokes equations) and Large Eddy Simulation (LES). However, the Reynolds averaging approach has a lower accuracy than LES [12]. LES is introduced in Section 2, and further detailed in C.

We denote $\langle \boldsymbol{u}(\boldsymbol{x},t)\rangle$ the mean velocity field. The velocity can then be decomposed into its mean and the fluctuation $\boldsymbol{u}'$ such as

$$\boldsymbol{u}(\boldsymbol{x},t) = \langle \boldsymbol{u}(\boldsymbol{x},t)\rangle + \boldsymbol{u}'(\boldsymbol{x},t). \tag{70}$$

This is referred to as the Reynolds decomposition. Taking the mean of the continuity equation (3) yields

$$\langle \nabla \cdot \boldsymbol{u}\rangle = \nabla \cdot \langle u\rangle = 0, \tag{71}$$

since derivation commutes with taking the mean [12]. Hence, the mean velocity is divergence free, and by subtraction so is the fluctuation. Then, taking the mean of the conservation of momentum (4b) yields

$$\frac{\partial \langle u_i\rangle}{\partial t} + \frac{\partial}{\partial x_j}\langle u_i u_j\rangle = -\frac{\partial \langle p\rangle}{\partial x_i} + \frac{1}{\mathrm{Re}}\frac{\partial}{\partial x_j}\left(\frac{\partial \langle u_i\rangle}{\partial x_j} + \frac{\partial \langle u_j\rangle}{\partial x_i}\right). \tag{72}$$

Hence the nonlinear convective terms yields the nonlinear mean $\langle u_i u_j\rangle$. Substituting the Reynolds decomposition for u_i and u_j yields

$$\begin{aligned}\langle u_i u_j\rangle &= \left\langle \left(\langle u_i\rangle + u_i'\right)\left(\langle u_j\rangle + u_j'\right)\right\rangle \\ &= \left\langle \langle u_i\rangle \langle u_j\rangle + u_i' \langle u_j\rangle + \langle u_i\rangle u_j' + u_i' u_j'\right\rangle \\ &= \langle u_i\rangle \langle u_j\rangle + \left\langle u_i' u_j'\right\rangle,\end{aligned} \tag{73}$$

since it can be shown that $\langle u_i' \langle u_j\rangle\rangle = 0$, that the mean is linear, and that the mean of the mean is the mean itself [12]. Then, substituting this back into (72) yields the Reynolds-Averaged Navier-Stokes equations

$$\frac{\partial \langle u_i\rangle}{\partial t} + \frac{\partial}{\partial x_j}\langle u_i\rangle\langle u_j\rangle = -\frac{\partial \langle p\rangle}{\partial x_i} + \frac{1}{\mathrm{Re}}\frac{\partial}{\partial x_j}\left(\frac{\partial \langle u_i\rangle}{\partial x_j} + \frac{\partial \langle u_j\rangle}{\partial x_i}\right) - \frac{\partial \left\langle u_i' u_j'\right\rangle}{\partial x_j}. \tag{74}$$

Equivalently, defining the Reynolds tensor $\tau_{ij} := \left\langle u_i', u_j'\right\rangle$, this can be rewritten as

$$\frac{\partial \langle \boldsymbol{u}\rangle}{\partial t} + \langle \boldsymbol{u}\rangle \cdot \nabla \langle \boldsymbol{u}\rangle = -\nabla \langle p\rangle + \frac{1}{\mathrm{Re}}\nabla^2 \langle \boldsymbol{u}\rangle - \nabla \cdot \boldsymbol{\tau}. \tag{75}$$

Reynolds stress models then aim to solve the closure problem that arises from these equations, namely by modelling the Reynolds stress tensor [12].

B Energy cascade

In this section we aim to provide a brief introduction and explanation to the concept of energy cascade, which is an important concept in the study of turbulent flows. Richardson first wrote about this notion in the 1920's, in which he summarised it by the following quote: *Big whirls have little whirls that feed on their velocity, and little whirls have lesser whirls, and so on to viscosity – in the molecular sense.* [13, p. 66]. This quotes highlights an important concept in turbulent flows, namely that such a flow can be considered as composed of eddies of different size. An eddy is viewed as a turbulent motion, located within a region of size ℓ. A region containing a large eddy can also contain smaller eddies [12].

Consider, for simplicity, a one-dimensional flow with very large Reynolds number and characteristic length and velocity scales $\mathcal{L}$ and $\mathcal{U}$. We denote the characteristic velocity of an eddy at scale ℓ by $u(\ell)$, and its characteristic timescale is $\tau := \ell/u(\ell)$. The largest eddies can be characterised by the length scale ℓ_0, which is of the order of $\mathcal{L}$. Their characteristic velocity is also of the order of the characteristic flow velocity

$$u_0 := u(\ell_0) \sim \mathcal{U}. \tag{76}$$

Therefore, these large eddies also have a large Reynolds number, that is

$$\mathrm{Re}_0 := \frac{u_0 \ell_0}{\nu} \sim \mathrm{Re}, \tag{77}$$

so hence the effect of viscosity on the large eddies is negligible. Richardson's concept is that these large eddies are unstable and therefore they break up into smaller eddies. But, these smaller eddies are also unstable, and so they break up into even smaller eddies, and so on. When the larger eddies break up into smaller eddies, energy is transferred. This process ends at the smallest scale, where the corresponding Reynolds number $\mathrm{Re}(\ell) = u(\ell)\ell/\nu$ is sufficiently small that viscous effects are not negligible anymore.

This transfer of energy between the scales is referred to as the *energy cascade*, and the smallest scales at which the eddy motion is stable are the *Kolmogorov scales*. They are the smallest scales in a turbulent flow, and at this scale the turbulent kinetic energy is dissipated into thermal energy [7][2]. The notion of energy cascade was first described by Richardson [13], and later formalised by Kolmogorov, who also formulated three hypothesis on this smallest scale [7]

B.1 Kolmogorov's first hypothesis

The rate at which energy is transferred between the scales is given by [12]

$$\varepsilon \sim \frac{u_0^3}{\ell_0}. \tag{78}$$

For sufficiently high Reynolds number, the small scale turbulent motions – i.e. with scale $\ell \ll \ell_0$ – are statistically isotropic, that is uniform in all directions. We denote the scale at which there is a demarcation between the anisotropic large eddies and the isotropic small eddies is denoted by ℓ_{EI}.

Kolmogorov's first similarity hypothesis [7, 12, 15] states that the statistics of the small-scales ($\ell < \ell_{EI}$) motions are uniquely determined by ε and ν. Using these two parameters, we can define unique (up to multiplication by a constant) lengths, time and velocity scles

$$\eta := \left(\frac{\nu^3}{\varepsilon}\right)^{1/4}, \tag{79a}$$

$$\tau_\eta := \left(\frac{\nu}{\varepsilon}\right)^{1/2}, \tag{79b}$$

$$u_\eta := (\varepsilon\nu)^{1/4}, \tag{79c}$$

which characterise the smallest dissipative eddies at the Kolmogorov scale. Computing the Reynolds number at this scale yields

$$\mathrm{Re}_\eta := \frac{\eta u_\eta}{\nu} = 1. \tag{80}$$

This highlights the fact that the larger eddies break up into smaller eddies and energy is transferred between the scales until the Reynolds number is sufficiently small for dissipation to come into effect. We can furthermore compute this rate of dissipation [12]

$$\varepsilon = \nu \left(\frac{u_\eta}{\eta}\right)^2 = \frac{\nu}{\tau_\eta^2}. \tag{81}$$

We can also compute the ratios of the characteristic length, velocity and time at the smallest scale with the largest scales, and write them in terms of the Reynolds number as

$$\frac{\eta}{\ell_0} \sim \mathrm{Re}^{-3/4}, \tag{82a}$$

$$\frac{u_\eta}{u_0} \sim \mathrm{Re}^{-1/4}, \tag{82b}$$

$$\frac{\tau_\eta}{\tau_0} \sim \mathrm{Re}^{-1/2}. \tag{82c}$$

It can then be seen that at high Reynolds number, the velocity and time scales at the smallest eddies, u_η and τ_η, are much smaller than those of the largest eddies, u_0, τ_0. Hence, as stated by Richardson, we can see that the larger eddies do break up into smaller eddies.

[2]This article is a translation of the original text in Russian published in 1941.

B.2 Kolmogorov's second hypothesis

The ratio of the smallest length scale to the largest (82b) scales as $\mathrm{Re}^{-3/4}$. This then means that the smallest scale scales as $\mathrm{Re}^{-3/4}\ell_0$, or in other words, for very high Reynolds number, there exists a range of scales such that $\eta \ll \ell \ll \ell_0$. Kolmogorov's second similarity theory [7, 12] states that the statics of the motions of scales in that range are uniquely determined by ε, independently of ν.

We can define separate ranges, namely [12]

- $\eta < \ell < \ell_{DI}$: dissipation range,
- $\ell_{DI} < \ell < \ell_{EI}$: inertial range,
- $\ell_{EI} < \ell < \ell_0 \sim \mathcal{L}$: energy containing range,

where the subscripts DI and EI indicate respectively that the scale is at the demarcation between dissipation (D) and inertial (I) ranges, and between the inertial and energy containing (E) ranges. Kolmogorov's 2$^{\text{nd}}$ hypothesis then applies to motions in the inertial range.

Only the motions in the dissipation range experience significant viscous effects, and are responsible for most (if not all) the dissipation. In the inertial subrange, the characteristic velocity and time scales are given by [12]:

$$u(\ell) = (\varepsilon\ell)^{1/3} \quad = u_\eta \left(\frac{\ell}{\eta}\right)^{1/3} \sim u_0 \left(\frac{\ell}{\ell_0}\right)^{1/3}, \tag{83a}$$

$$\tau(\ell) = \left(\frac{\ell^2}{\varepsilon}\right)^{1/3} = \tau_\eta \left(\frac{\ell}{\eta}\right)^{2/3} \sim \tau_0 \left(\frac{\ell}{\ell_0}\right)^{2/3}. \tag{83b}$$

Then, a consequence of this is that in the inertial range, both the velocity and time scales decrease as ℓ increases. Hence, as the length scale approaches the energy containing range, the velocity and time scales of the motion decrease. Another consequence of equations (83a) and (83b) is that we can express the transfer rate of energy in the inertial range as

$$\varepsilon = \frac{u(\ell)}{\tau(\ell)}, \tag{84}$$

meaning that in this range, the rate at which energy is transferred is independent of (or at least not directly dependent on) the length scale.

It can also be shown that the rate at which energy is transferred between the scales scales as ε (84) for all aforementioned ranges [12]. Hence, the rate of energy transfer between the larger scales in the energy containing range determines the rate of energy transfer in the inertial range, which also determines the rate of energy transfer in the dissipation range. This then determines the rate of energy dissipation into heat (78).

B.3 Energy spectrum

In the previous section, we have found a relation between the energy transfer rate at the different scales. We still need a way to express how the turbulent kinetic energy is distributed among eddies of different sizes. This is done by considering an energy spectrum function $E(\kappa)$. In the inertial range, motions at a length scale ℓ correspond to wavenumber $\kappa = 2\pi/\ell$. The energy in the wavenumber range (κ_a, κ_b) is given by [12]:

$$k_{(\kappa_1,\kappa_b)} := \int_{\kappa_a}^{\kappa_b} E(\kappa)\,\mathrm{d}\kappa, \tag{85}$$

and the turbulent kinetic energy is found as an integral over the whole spectrum [15]

$$K := \int_0^\infty E(\kappa)\,\mathrm{d}\kappa, \tag{86}$$

where the same results in higher spatial dimension.

Under the assumption of local isotropy, we can find forms of the energy spectrum function in the aforementioned ranges of scales of motion.

- In the energy containing range, i.e. at a large scale, there is no universal form for the energy spectrum, but it can be said that [15]
$$E(\kappa) \approx \kappa^4 \text{ or } E(\kappa) \approx \kappa^2 \text{ for } \kappa \ll 1. \tag{87}$$
- In the inertial range, as shown previously, ε is independent of the length scale ℓ and the energy spectrum depends only on κ and ε, so it can be written as [12, 15]
$$E(\kappa) = C\varepsilon^{2/3}\kappa^{-5/3}, \tag{88}$$
where C is the Kolmogorov constant. After many experiments, it may be assumed that $C = 1.5$ [17].
- In the dissipation range, where the energy is dissipated by the effect of viscosity, the energy spectrum depends explicitly only on κ, ν and ε, which can also be rewritten so that $E(\kappa)$ depends only on κ, η, u_η. No dimensional argument allows to find a unique form for $E(\kappa)$ in this range, but some arguments may suggest to have an exponential decay of the energy spectrum in the dissipation range [15].

The energy spectrum in the inertial range (88) is called the Kolmogorov $-\frac{5}{3}$ law, named as such after the exponent of the wavenumber [12].

C More details on Large Eddy Simulation

In this section we aim to provide a more detailed explanation of the steps and the approach undertaken in Large Eddy Simulation. This section is meant as an addition to Section 2.

C.1 Examples of filters

As mentioned in Section 2.1, in the paper we consider an implicit filter given by the grid discretisation. In this section, we give three examples of filters: the box filter, the spectral or sharp cutoff filter, and the Gaussian filter. All these filters are here looked into for a one-dimensional domain, but they can be expanded to higher spatial dimensions. We focus on filters independent of x. In one dimension, where the fluid domain is the whole real line, the filtered velocity is given by the convolution
$$\overline{u} = \int_{-\infty}^{\infty} G(r)u(x-r)\,\mathrm{d}r. \tag{89}$$
To properly see the effects of the filter on the velocity field, we turn to Fourier space. Suppose $u(x)$ has Fourier transform
$$\hat{u}(\kappa) := \mathcal{F}\{u(x)\}, \tag{90}$$
then the Fourier transform of the filtered velocity is, by properties of Fourier transforms of convolutions,
$$\hat{\overline{u}}(\kappa) := \mathcal{F}\{\overline{u}(x)\} = \hat{G}(\kappa)\hat{u}(\kappa), \tag{91}$$
where the transfer function $\hat{G}(\kappa)$ is 2π times the Fourier transform of the filter
$$\hat{G}(\kappa) := \int_{-\infty}^{\infty} G(r)e^{-i\kappa r}\,\mathrm{d}r = 2\pi\mathcal{F}\{G(r)\}. \tag{92}$$
By the normalisation considitions it also holds that
$$\hat{G}(0) = \int_{-\infty}^{\infty} G(r)\,\mathrm{d}r = 1. \tag{93}$$
When looking at the three aforementioned examples of filters, we consider both the filtering operator and it's Fourier transform as given in Table 5. All filters are determined by the filter length Δ, to be taken $\Delta \leq \ell_{EI}$ ideally. Note that in the box and sharp cutoff filter, H denotes the Heaviside step function [2]
$$H(x) := \begin{cases} 1, & \text{if } x \geq 0, \\ 0, & \text{if } x < 0. \end{cases} \tag{94}$$

Filter	$G(r)$	$\hat{G}(\kappa)$
Box filter	$\frac{1}{\Delta}H(\frac{1}{2}\Delta - \lvert r\rvert)$	$\dfrac{\sin(\kappa\Delta/2)}{\kappa\Delta/2}$
Sharp cutoff filter	$\dfrac{\sin(\pi r/\Delta)}{\pi r}$	$H(\kappa_c - \lvert\kappa\rvert)$
Gaussian filter	$\left(\dfrac{6}{\pi\Delta^2}\right)^{1/2}\exp\left(-\dfrac{6r^2}{\Delta^2}\right)$	$\exp\left(-\dfrac{\kappa^2\Delta^2}{24}\right)$

Table 5. Three filters for Large Eddy Simulation: box, sharp cutoff and Gaussian [12, 15].

Moreover, in the sharp cutoff, κ_c denotes the cutoff wavenumber, $\kappa_c := \pi/\Delta$.

The energy spectrum (Appendix B.3) of the filtered velocity can be written in terms of the energy spectrum of the velocity, once again by using properties of Fourier transform [12]:

$$\overline{E}(\kappa) = \hat{G}(\kappa)^2 E(\kappa). \tag{95}$$

C.2 The Smagorinsky model

In the following, we consider turbulent flows with high Reynolds numbers, and we suppose that the filter width Δ is in the inertial range $\ell_{DI} < \Delta < \ell_{EI}$ (see Appendix B). We also know that in the inertial range, the energy spectrum $E(\kappa)$ is in Kolmogorov $-\frac{5}{3}$ form (88). The Smagorinsky model is the simplest model for the anisotropic residual stress tensor τ_{ij}^r (12) [12]. Before looking into this model, we briefly go back to the unfiltered Navier-Stokes equations presented in Section 1.2.

C.2.1 Rate of strain

The term $\frac{\partial u_i}{\partial x_j}$ from the conservation of momentum (4b) can be decomposed into its symmetric-deviatoric and antisymmetric parts

$$\frac{\partial u_i}{\partial x_j} = S_{ij} + \Omega_{ij}, \tag{96}$$

where in the case of a variable-density flow the right hand side of (96) also contains the isotropic term $\frac{1}{3}\delta_{ij}\nabla\cdot\boldsymbol{u}$, which is nonzero for compressible flows. In the above equation (96), S_{ij} is the symmetric deviatoric rate of strain tensor and Ω_{ij} is the antisymmetric rate of rotation tensor

$$S_{ij} := \frac{1}{2}\left(\frac{\partial u_i}{\partial x_j} + \frac{\partial u_j}{\partial x_i}\right), \tag{97a}$$

$$\Omega_{ij} := \frac{1}{2}\left(\frac{\partial u_i}{\partial x_j} + \frac{\partial u_j}{\partial x_i}\right). \tag{97b}$$

The rate of rotation can be related to the vorticity, given in more detail in [12]. The Smagorinsky model then used the filtered rate of strain, as can be seen below.

C.2.2 The eddy-viscosity model

The goal of the Smagorinsky model is to get closure for the conservation equations of the filtered velocity (14). This model consists of two parts [12, 15].

1. We define the linear eddy-viscosity model

$$\tau_{ij}^r = -2\nu_r \overline{S}_{ij}, \tag{98}$$

which relates the residual stress τ_{ij}^r to the filtered rate of strain $\overline{S}_{ij}$. The proportionality coefficient $\nu_r(\boldsymbol{x}, t)$ is the eddy viscosity of the subgrid scale motions. This first step is common to all eddy-viscosity models.

2. This eddy viscosity is modelled by
$$\nu_r = \ell_s^2 \overline{\mathcal{S}} = (C_s \Delta)^2 \overline{\mathcal{S}}, \tag{99}$$
where ℓ_s is the Smagorinsky lengthscale and C_s the Smagorinsky coefficient such that $\ell_s = C_s \Delta$.

The Smagorinsky constant depends on the type of flow considered and is to be measured experimentally. A theoretical value was found by Lilly to be $C_s = 0.18$, given the Kolmogorov constant $C \approx 1.5$, but for instance in shear flow, one may prefer considering $C_s = 0.1$ [1, 9]. This lack of universal constant is one of the drawbacks of the Smagorinksy model [5]

C.2.3 Characteristic filtered rate of strain

The filtered rate of strain is obtained similarly to the rate of strain (97a), and is given by
$$\overline{S}_{ij} := \frac{1}{2}\left(\frac{\partial \overline{u}_i}{\partial x_j} + \frac{\partial \overline{u}_j}{\partial x_i}\right). \tag{100}$$

The characteristic filtered rate of strain is defined as
$$\overline{\mathcal{S}} := \left(2\overline{S}_{ij}\overline{S}_{ij}\right)^{1/2}. \tag{101}$$

Its mean square can be written in terms of the energy spectrum and transfer function [12]
$$\begin{aligned} \left\langle \overline{\mathcal{S}}^2 \right\rangle &= 2\left\langle \overline{S}_{ij}\overline{S}_{ij} \right\rangle \\ &= 2\int_0^\infty \kappa^2 \overline{E}(\kappa)\,\mathrm{d}\kappa \\ &= 3\int_0^\infty \kappa^2 \hat{G}(\kappa)^2 E(\kappa)\,\mathrm{d}\kappa, \end{aligned} \tag{102}$$

where for the last equality we use the relation between the filtered and unfiltered energy spectra (95). We assume the energy spectrum is in the Kolmogorov $-\frac{5}{3}$ form. Furthermore, we can assume that the inertial range is larger than the other ranges, and that outside this range the energy spectrum has a sharp decrease [12]. Hence, the integral can be approximated as
$$\begin{aligned} \left\langle \overline{\mathcal{S}}^2 \right\rangle &\approx \int_0^\infty \kappa^2 \hat{G}(\kappa)^2 C\varepsilon^{2/3}\kappa^{-5/3}\,\mathrm{d}\kappa \\ &= a_f C \varepsilon^{2/3}\Delta^{-4/3}, \end{aligned} \tag{103}$$

where C is the Kolmogorov constant (which can be taken as $C = 1.5$) and the constant
$$a_f := 2\int_0^\infty (\kappa\Delta)^{1/3}\hat{G}(\kappa)^2 \Delta\,\mathrm{d}\kappa \tag{104}$$

depends only on the filter chosen, and is independent of the filter length Δ. This constant can be evaluated for the three filters considered in Appendix C.1, and is given in Table 6.

Filter	a_f
Box filter	$-2\Gamma(-\frac{2}{3}) \approx 8$
Sharp filter	$\frac{3}{4}\pi^{4/3} \approx 6.90$
Gaussian filter	$12^{2/3}\Gamma(\frac{2}{3}) \approx 2.10$

Table 6. Values of a_f (104) for the filters considered in Appendix C.1: box, sharp cutoff and Gaussian.

C.2.4 Transfer rate of subgrid scale kinetic energy

Recall that the residual kinetic energy $k_r = \frac{1}{2}\tau_{ii}^R$ was absorbed in the modified pressure, as stated in Section 2.2. Then, from the anisotropic residual stress tensor we can obtain the rate of production of subgrid scale kinetic energy

$$\mathcal{P}_r := -\tau_{ij}^r \overline{S}_{ij}. \tag{105}$$

Substituting the eddy viscosity model (98) yields

$$\mathcal{P}_r = 2\nu_r \overline{S}_{ij}\overline{S}_{ij} = \nu_r \overline{\mathcal{S}}^2. \tag{106}$$

For the Smagorinsky model, and in general for any eddy viscosity model with $\nu_r > 0$, there is no backscatter modelled [12]. This is another drawback of such a model, since for more accurate modelling it is best to also model the backscatter of energy [1, 5]. However, the fact that there is no backscatter here means that the the energy transfer (106) is everywhere the same, from the filtered to the residual motions.

Taking the mean of (106) and substituting the Smagorinsky eddy viscosity (99) yields

$$\langle \mathcal{P}_r \rangle = \left\langle \nu_r \overline{\mathcal{S}}^2 \right\rangle = \ell_s^2 \left\langle \overline{\mathcal{S}}^3 \right\rangle. \tag{107}$$

Therefore, recalling the mean of the characteristic filtered rate of strain (103) and rewriting yields

$$\ell_s = \frac{\Delta}{(Ca_f)^{3/4}} \left(\frac{\left\langle \overline{\mathcal{S}}^3 \right\rangle}{\left\langle \overline{\mathcal{S}}^2 \right\rangle^{3/2}} \right)^{-1/2}. \tag{108}$$

We can approximate, for any p not too large, $\langle \overline{\mathcal{S}}^p \rangle \approx \langle \overline{\mathcal{S}}^2 \rangle^{p/2}$ [12], which then allows the simplifications of the Smagorinky length ℓ_s (108). Moreover, this approximation and the above results allow to find estimates of filtered and residual quantities for specific filters in the inertial range. Pope describes those quantities for the sharp cutoff filter [12, Table 13.3], and we derived those same quantities for the box filter, which can be found in Table 7.

Quantity	Normalised quantity	Estimate of normalised quantity
Residual kinetic energy	$\frac{\langle k_r \rangle}{k}$	$0.9C\left(\frac{\Delta}{L}\right)^{2/3}$
Rate of transfer to the residual motions	$\frac{\langle \mathcal{P}_r \rangle}{\varepsilon}$	1
Dissipation from filtered motions	$\frac{\langle \varepsilon_f \rangle}{\varepsilon}$	$8C\left(\frac{\Delta}{\eta}\right)^{-4/3}$
Filtered rate of strain	$\frac{\langle \overline{\mathcal{S}}^2 \rangle^{1/2} k}{\varepsilon}$	$2\sqrt{2C}\left(\frac{\Delta}{L}\right)^{-2/3}$
Residual stress	$\frac{\langle \tau_{ij}^r \tau_{ij}^r \rangle^{1/2}}{k}$	$\frac{1}{\sqrt{2c}}\left(\frac{\Delta}{L}\right)^{2/3}$
Residual eddy viscosity	$\frac{\langle \nu_r \rangle \varepsilon}{k^2}$	$\frac{1}{8C}\left(\frac{\Delta}{L}\right)^{4/3}$
Smagorinsky lengthscale	$\frac{\ell_s}{L}$	$0.2C^{-3/4}\frac{\Delta}{L}$

Table 7. Estimates of filtered and residual quantities for the box filter in the inertial subrange of high Reynolds number turbulence. The quantities are normalised by k, ε and $L := k^{3/2}/\varepsilon$. We may consider the Kolmogorov constant as $C = 1.5$.

C.2.5 The Smagorinsky filter

For a homogeneous isotropic turbulent flow, the Smagorinsky filter is the only filter that is uniquely determined by the filter length Δ satisfying the following transfer function [12]

$$\hat{G}(\kappa) = \left(\frac{\overline{E}(\kappa)}{E(\kappa)}\right)^{1/2}. \tag{109}$$

At high Reynolds numbers, and for characteristic length ℓ_s in the inertial range, this filter can be computed as

$$\hat{G}(\kappa) = \exp\left\{-\frac{3}{4}C\kappa^{4.3}\left(\overline{\eta}^{4/3} - \eta^{4/3}\right)\right\}, \tag{110}$$

for the Kolmogorov effective scale

$$\overline{\eta} := \left(\frac{(\nu+\nu_r)^3}{\varepsilon}\right)^{1/4} = \ell_s\left(1+\frac{\nu}{\nu_r}\right)^{1/2}, \tag{111}$$

and η the length scale of the smallest motions (79a) (see Appendix B.1).

D Additional plots

In this section we present additional plots that were not added to the main body of this paper. The sections refer to the sections in which these plots are referred to, or sections for which these plots are relevant.

D.1 Method of Manufactured Solutions

The plots in this section follow the same structure as those in Section 7.2, Figure 12, which is as follows. the top left plot shows the regular mesh discretisation. The top right plot shows the evolution of the norm of the divergence of the velocity field over time. The bottom left plot shows the norm difference $\|\boldsymbol{u} - \boldsymbol{u}_0\|_2$ taken at cell centres and normalised by the number of cells. The bottom right plot shows the norm difference $\|p - p_0\|_2$ also taken at cell centres and also normalised. This normalisation is done by dividing the Euclidean norm by the square root of the number of cells. The value to which these norms converge are indicated in the legend of each figure. For completeness, the plots of $N = 12, 24$ are also added here, to allow for a better and clearer visualisation of the convergence behaviour. For $N = 384$, we do not plot the grid since the increase in resolution is not visible compared to $N = 192$.

$Re = 1600,\ N = 6,\ dt = 0.16667,\ dx = 0.16667$

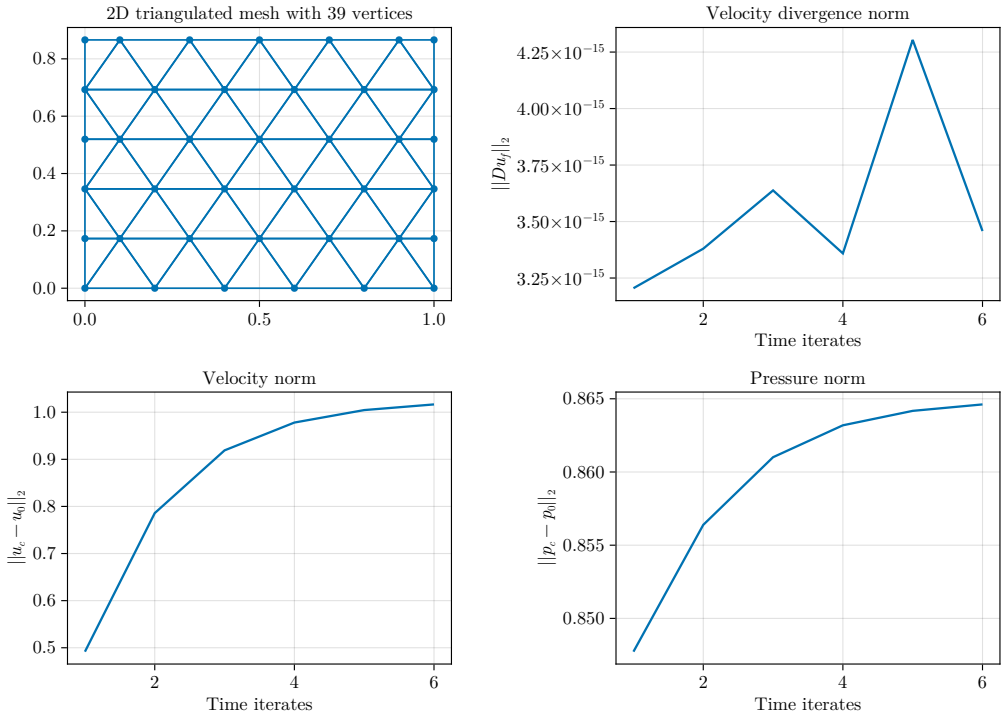


Figure 16. $N = 6,\ \|\boldsymbol{u} - \boldsymbol{u}_0\| \to 1.0165,\ \|p - p_0\| \to 0.86462.$

$Re = 1600,\ N = 12,\ dt = 0.08333,\ dx = 0.08333$

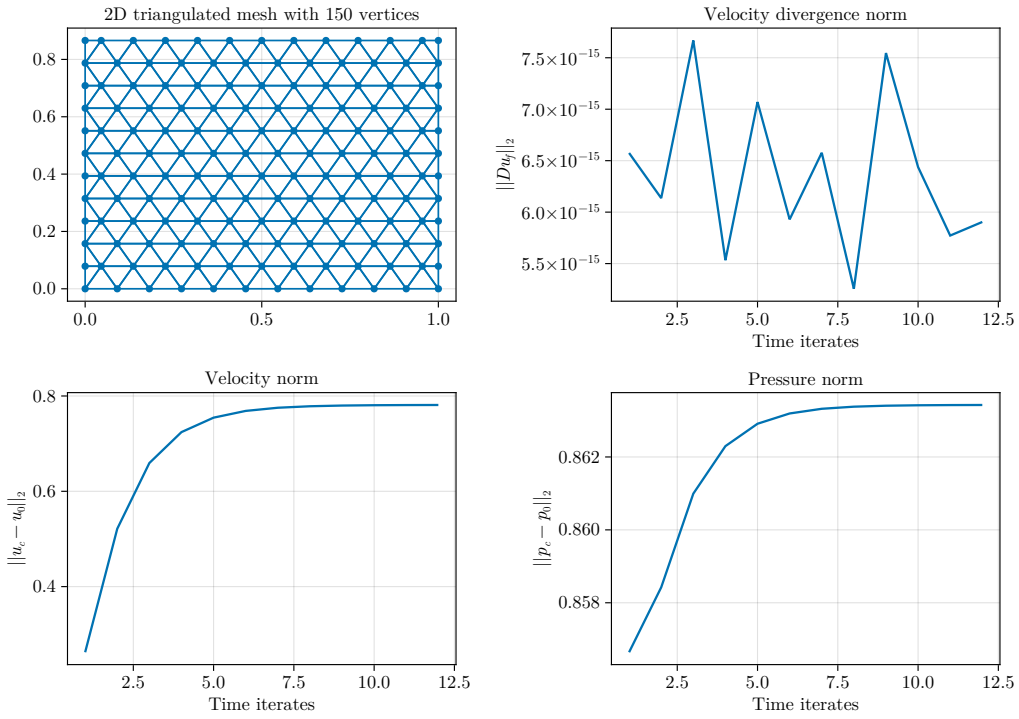


Figure 17. $N = 12,\ \|\boldsymbol{u} - \boldsymbol{u}_0\| \to 0.78111,\ \|p - p_0\| \to 0.86343.$

$Re = 1600,\ N = 24,\ dt = 0.04167,\ dx = 0.04167$

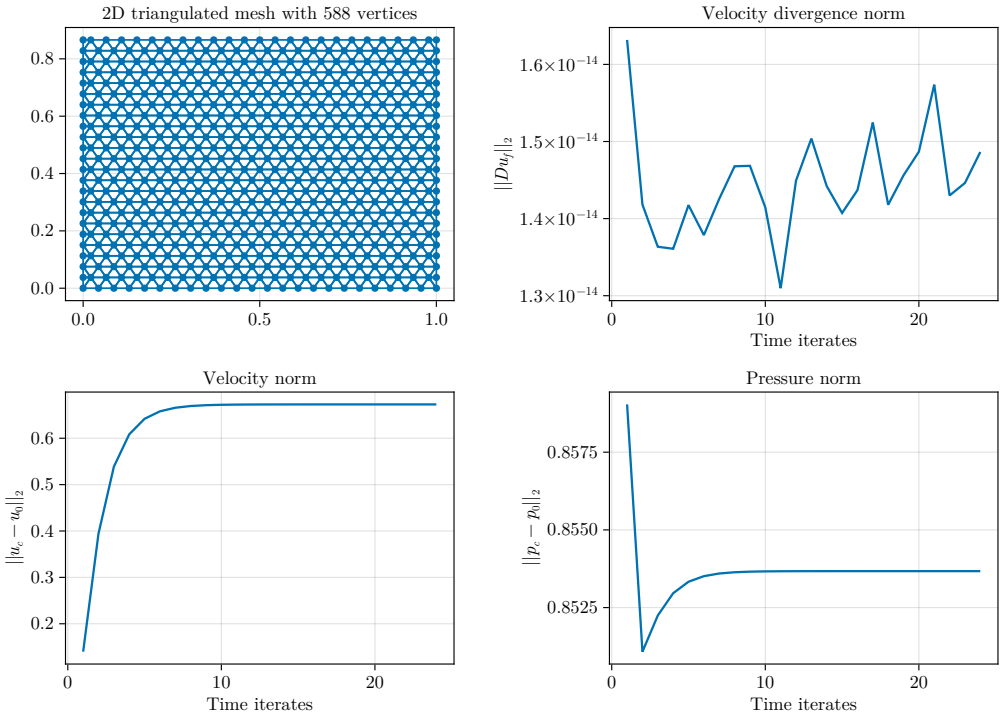


Figure 18. $N = 24,\ \|\boldsymbol{u} - \boldsymbol{u}_0\| \to 0.67291,\ \|p - p_0\| \to 0.85367.$

$Re = 1600,\ N = 48,\ dt = 0.02083,\ dx = 0.02083$

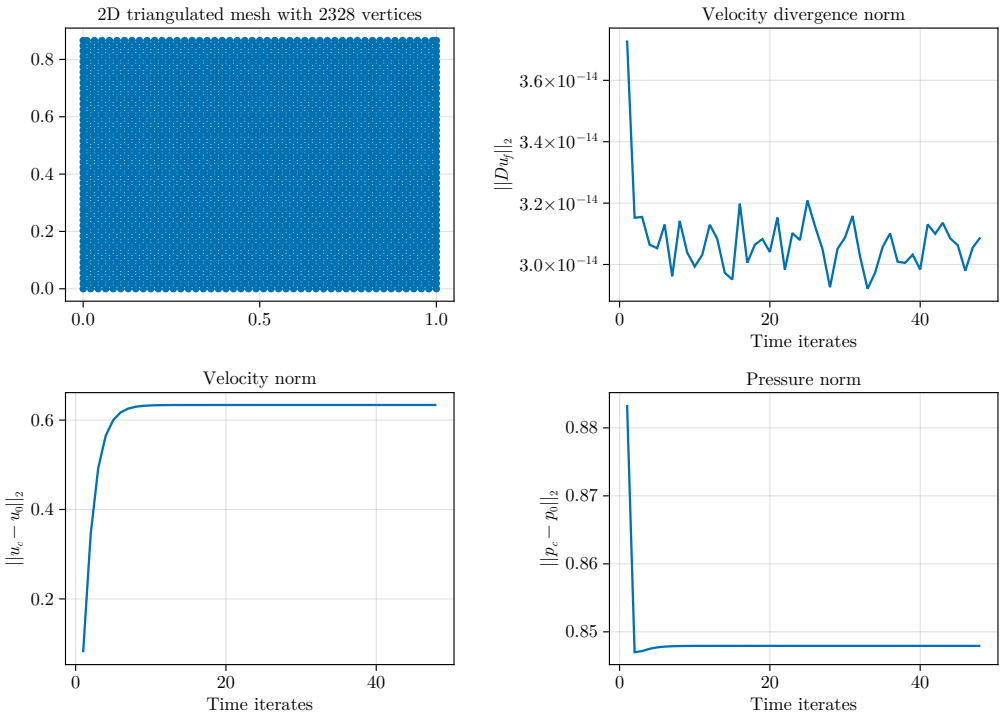


Figure 19. $N = 48,\ \|\boldsymbol{u} - \boldsymbol{u}_0\| \to 0.63373,\ \|p - p_0\| \to 0.84794.$

$Re = 1600,\ N = 96,\ dt = 0.01042,\ dx = 0.01042$

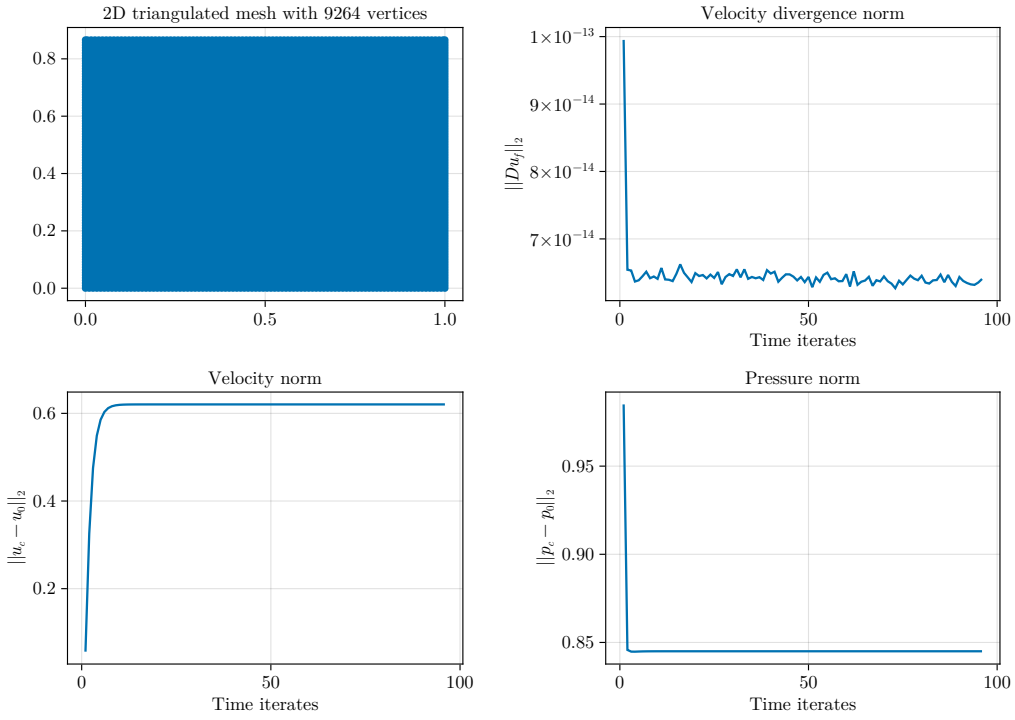


Figure 20. $N = 96,\ \|\boldsymbol{u} - \boldsymbol{u}_0\| \to 0.62042,\ \|p - p_0\| \to 0.8450.$

$Re = 1600,\ N = 192,\ dt = 0.00521,\ dx = 0.00521$

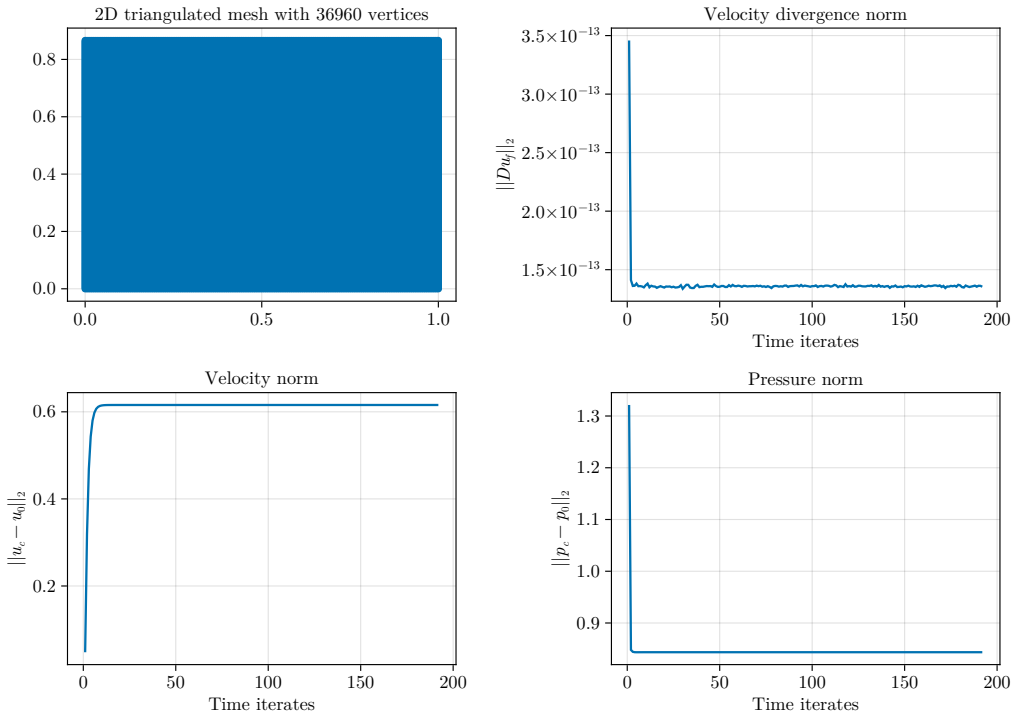


Figure 21. $N = 196,\ \|\boldsymbol{u} - \boldsymbol{u}_0\| \to 0.61569,\ \|p - p_0\| \to 0.84353.$

$Re = 1600,\ N = 384,\ dt = 0.0013,\ dx = 0.0026$

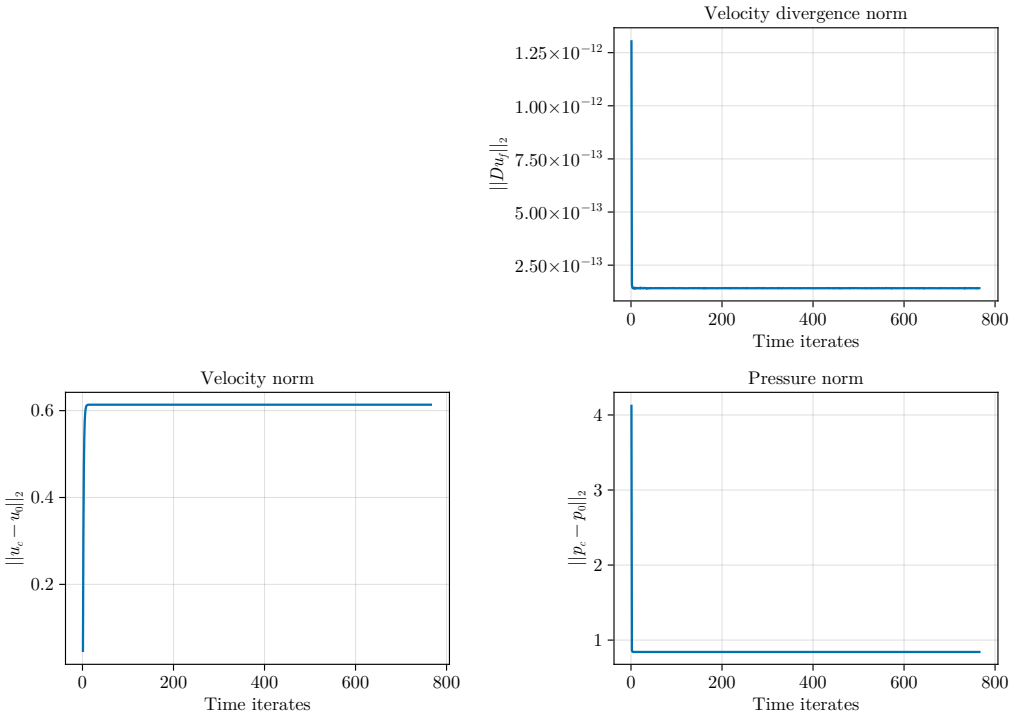


Figure 22. $N = 384$, $\|\boldsymbol{u} - \boldsymbol{u}_0\| \to 0.61569$, $\|p - p_0\| \to 0.84353$.